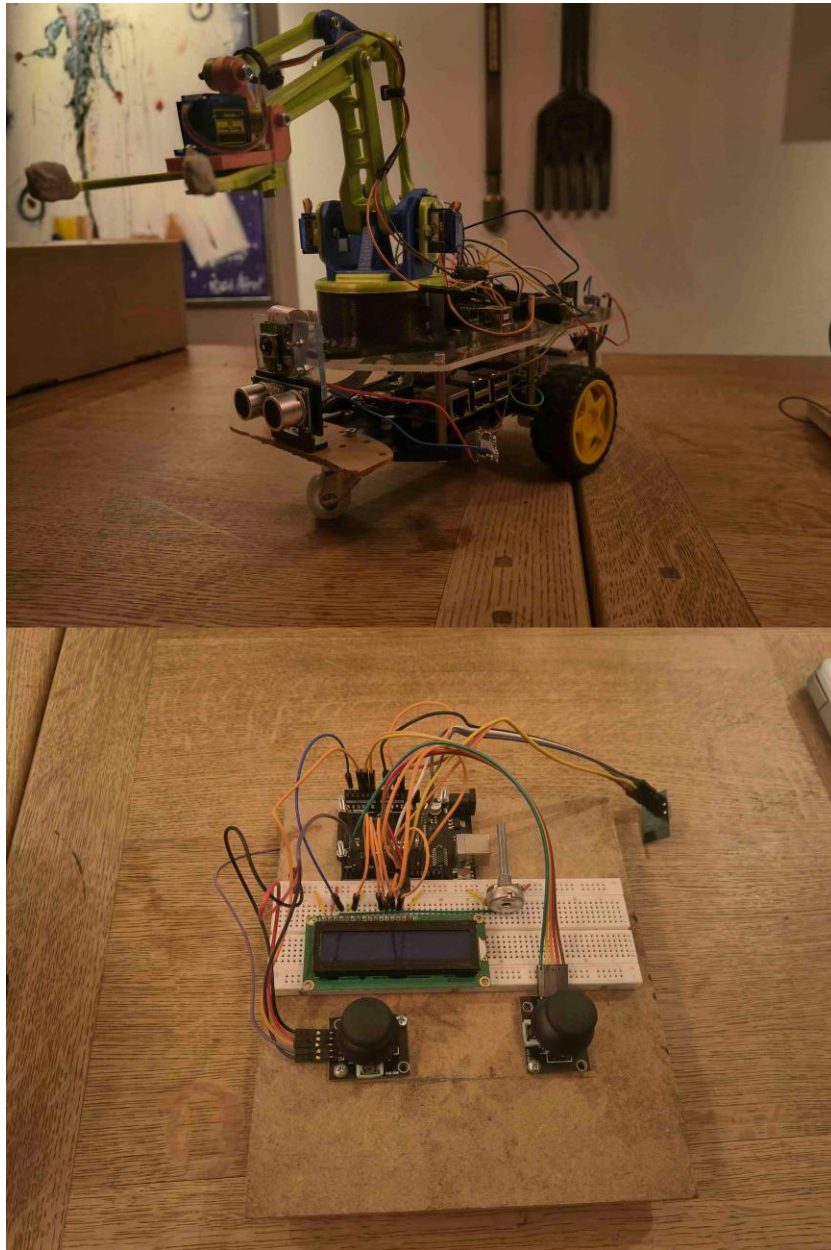


Rover Projektrapport

Projekttitel	Rover
Gruppemedlemmer	Kalle Egelund Bloch Nicklas Klingenberg Vejs Rinus Senker Tvede Julian Johansson Rasmussen
Vejleder	Daniel Kolditz Rubin-Grøn Palle Zych Tomasz Blaszczyk Zuhair Haroon Khan
Uddannelse og semester	IT-teknolog - 2. semester
Dato	02-05-2026
Institution	Zealand

Antal anslag: 75533

Rover til månen



1. Resumé (Julian)

Denne rapport omhandler videreudviklingen af en rover til NASA der har flere funktioner inden for automatisering og sensorteknologi. Roveren er designet til at kunne bevæge sig selvstændigt rundt samt udføre simple opgaver ved hjælp af en robotarm.

Robotarmen gør det muligt for roveren at både samle og placere genstande. Dette giver roveren mulighed for at udføre praktiske opgaver, hvor objekter skal flyttes og håndteres.

For at undgå sammenstød er roveren udstyret med en afstand sensor, som måler afstanden til objekter foran den. Sensoren hjælper roveren med at registrere forhindringer og ændre retning, så den kan navigere mere sikkert i omgivelserne.

Derudover indeholder roveren en DHT22-sensor, som bruges til at måle både temperaturen og luftfugtigheden. Disse målinger kan anvendes til overvågning af miljøet og giver roveren ekstra funktionalitet inden for dataindsamling.

Roverprojektet er en videreudvikling af et ældre roverprojekt som omhandlede at en rover kunne styres via en controller imens den kunne samle objekter op og ned. I den videreudviklet version skal den kunne styres manuelt via en server, teste for luftfugtighed og temperatur, bruge AI (objektgenkendelse/farvegenkendelse), samt automation og en ultrasonic sensor til at måle afstanden.

Rapporten beskriver både konstruktionen, programmering og testen af roverens forskellige komponenter samt hvordan sensorer og robotarm arbejder sammen for at skabe et funktionelt system.

Indholdsfortegnelse:

1. Resumé (Julian)	3
2. Indledning	6
2.1 Baggrund (Julian).....	6
2.2 Problemformulering (Alle):	6
2.3 Formål og afgrænsning af problemformulering (Julian).....	7
3. Metode	7
3.1 Værktøjer og teknologier (Julian)	7
3.2 Scrum (Rinus)	8
3.2.1 Interessentanalyse (Rinus).....	8
4. Analyse	10
4.1 Kravspecifikation (Alle).....	10
4.1.1 Funktionelle krav	10
4.1.2 Ikke-funktionelle krav	10
4.2 Hardware- og softwarevalg	11
4.2.1 Hardwarevalg (Julian)	11
4.2.2 Softwarevalg (Rinus).....	16
4.3 Softwarearkitektur	18
4.3.1 Frontend (Kalle)	19
4.3.2 Backend (Kalle).....	21
4.3.3 PiCore (Nicklas)	32

4.3.4 Arduino firmware (Nicklas)	34
4.4 Kommunikation.....	35
4.4.1 NRF24L01 radio (Nicklas).....	35
4.4.3 WebSocket (Nicklas).....	36
4.4.4 Seriel forbindelse mellem Raspberry Pi og Arduino (Rinus).....	40
4.5 Computer Vision og AI.....	43
4.5.1 OpenCV (Rinus).....	43
4.5.2 Objektdetektion (Rinus).....	45
4.5.3 Autonomous Mode (Nicklas)	47
4.6 Database (Nicklas)	48
4.6.1 MariaDB	48
5. Dokumentation af udviklingsproces	50
5.1 Implementering og fejlretning.....	50
5.2 Hardwareopsætning (Nicklas).....	56
5.3 Objektorienteret Programmering (Nicklas).....	57
5.5 Brug af AI i projektet (Alle).....	62
6. Konklusion (Alle)	63
6.1 Erfaringer	63
6.2 Diskussion	63
6.3 Fremtidige forbedringer	65
6.4. Konklusion	65
7. Kildeliste:.....	66

8. Bilag:.....	68
Bilag A – Scrum-sprints dokumentation (vedhæftet i separat PDF)	68
Bilag B - Backlog (vedhæftet i separat PDF) Bilag C - Github - https://github.com/Klaes25/examproject	68

2. Indledning

2.1 Baggrund (Julian)

Projektet foregår på IT-teknolog uddannelsen og er sammenfattet af 4 fag indlejrede systemer, netværksteknologi, programmering og projektstyring og forretningsforståelse.

Roveren er udstyret med en robotarm, DHT22 sensor, ultrasonic sensor HC-SR04 og et kamera hvilket gør det muligt for roveren ikke kun at køre men også manipulere objekter, måle luftfugtighed og temperatur, sikre sig at undgå at støde ind i andre objekter og ved hjælp af en raspberry pi3 gøre det muligt at styre roveren manuelt fra en server.

Roveren er en videreudvikling fra et ældre projekt med en robotarm, hvor der bliver tilføjet DHT22 sensor, ultrasonic sensor HC-SR04 og et kamera så man kan indsamle data ind for NASA fra de andre planeter.

2.2 Problemformulering (Alle):

Hvordan kan en manuelt styret rover videreudvikles til et system med både web baseret manuel styring og autonom navigation, og hvordan kan ultrasonic sensor, kamera, backend og frontend kontrolpanel integreres i et eksisterende roversystem?

2.3 Formål og afgrænsning af problemformulering (Julian)

Formålet er at komme med en solid rover der har flere funktionalitet, som ikke kun gør det både nemmere og mere brugervenligt men også gør det muligt at indsamle flere forskellige slags data ind på en gang i stedet for at bruge flere slags maskiner til det. Roverens egnet brug er at indsamle data ind til NASA fra fx. Månen så man kan have en bedre viden inden for miljøforholdene på andre planeter.

3. Metode

3.1 Værktøjer og teknologier (Julian)

Hardware:

- motorcontroller – l298n.
- 4 x servomotor – sg90.
- DHT22.
- ultrasonic sensor HC-SR04.
- Nrf24l01.
- 2 x step down buck converter 4.5-24 dc-dc 3A 0.8-17V.
- raspberry pi3 kamera.
- Raspberrypi3.
- arduino uno.
- strøm forsyning 2x 3,7v seriel (7,4v) i alt 6000mAH.
- 2 x Mini DC gearmotor - 3-6Vdc / 200mA.
- 16x2 LCD display modul.
- 10k ohm potentiometer.

3.2 Scrum (Rinus)

Vi har arbejdet ud fra Scrum, som er en agil metode der egner sig bedst til projekter med skiftende og usikre krav. Det er en iterativ tilgang, hvilket betyder at udvikling og forbedring af produktet sker i korte cyklusser med fokus på feedback. Man arbejder i sprints, som er korte tidsperioder, hvor man tager sig af bestemte dele af projektet og justerer løbende indsatsen på baggrund af feedback. Efter hvert sprint er der et møde hvor man reflekterer over forbedringer til næste sprint, hvilket giver viden om hvad der skal prioriteres i projektet, hvor meget tid og resurser der skal bruges, på de forskellige dele.

Et Scrum team består af en Product Owner, en Scrum Master og Developers, som hver har deres eget ansvarsområde. Product Owner repræsenterer interessenterne og er ansvarlig for at prioritere opgaverne i backloggen. Scrum Master sikrer at teamet arbejder efter Scrum-principperne og fjerner forhindringer der kan bremse arbejdet. Developers er dem der udfører det egentlige arbejde og leverer et færdigt produkt ved afslutningen af hvert sprint.¹

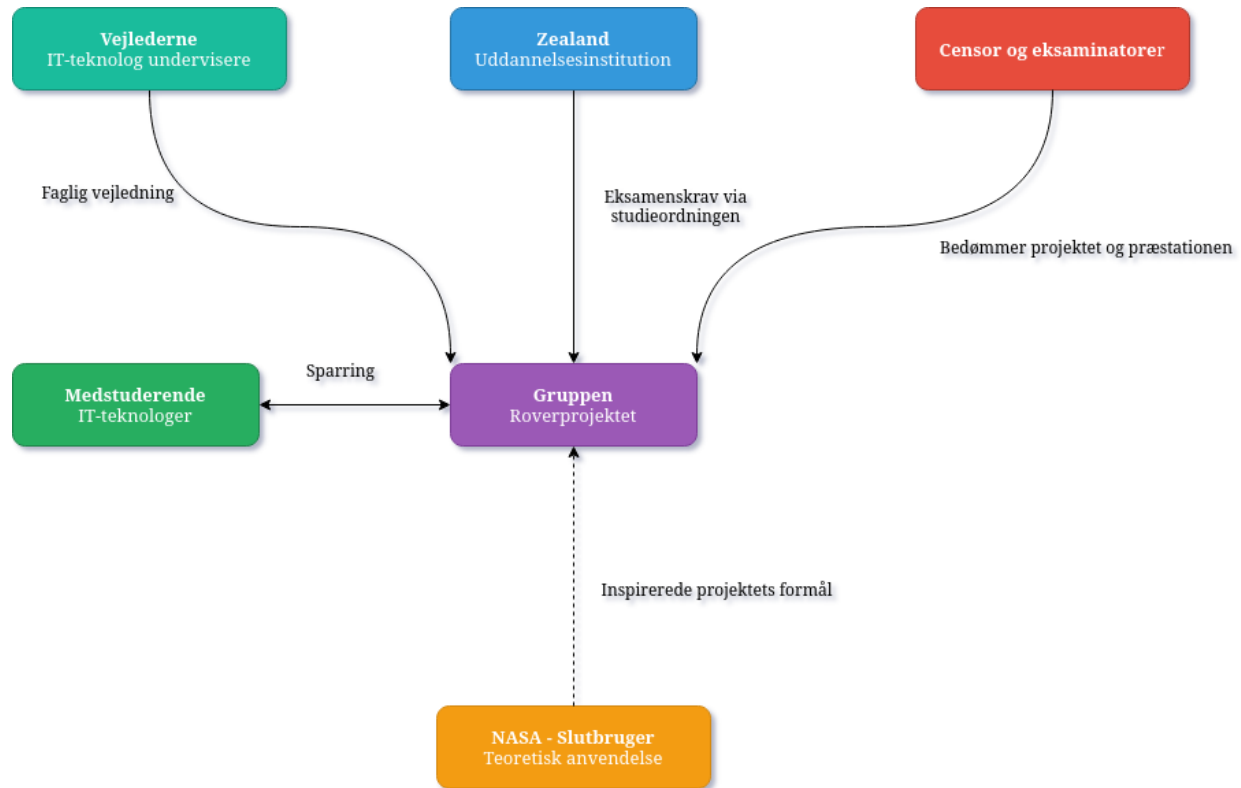
3.2.1 Interessentanalyse (Rinus)

Som en del af projektstyringen udarbejdede vi en interessentanalyse, hvor vi analyserede hvilke interessenter, havde en indflydelse på projektet og hvordan. Pilene repræsenterer retningen af indflydelsen, og den stiplede linje er en indikation af indirekte indflydelse.

Vejlederne og Zealand påvirker projektet gennem faglig feedback samt de eksamenskrav der var defineret i vores uddannelses studieordning. Censor og eksaminatorerne bedømmer det færdige

¹ <https://www.mannaz.com/da/artikler/projektledelse-agile-principper/hvad-er-scrum-en-introduktion-til-scrum-metoden/>

produkt og den mundtlige præstation. Medstuderende bidrager gennem sparring, og NASA fungerer som en slags slutbruger i den teoretiske sans, som har inspireret vores valg i projektet.



4. Analyse

4.1 Kravspecifikation (Alle)

4.1.1 Funktionelle krav

Når vi skulle designe vores roverprojekt, havde vi i den viden at der var en række funktionelle krav som var essentielt at have for at kunne fange data, bruge robotarmen, måle afstanden via en afstandssensor osv. Funktionelle krav er de handlinger, funktioner, og opgaver, som systemet skal kunne og må kunne udføre.

1. Roveren skal kunne styres manuelt via en server.
2. Roveren skal kunne vise en direkte transmission fra et kamera til serveren.
3. Roveren skal have fuld funktionalitet af robotarmen så den kan manipulere objekter.
4. Roverens sensor skal kunne måle temperatur og fugtighed udenfor via en DHT22 sensor.
5. Roveren skal kunne undgå sammenstød via ultrasonic sensor.
6. Roveren skal kunne bruge AI til objektgenkendelse og farvegenkendelse.

4.1.2 Ikke-funktionelle krav

Der er også ikke-funktionelle krav, som handler om, hvordan roverens system skal fungere i stedet for hvad roverens system, skal kunne gøre.

1. Kommunikation mellem Pi og server skal foregå over en sikker forbindelse via reverse proxy.
2. Systemet skal kræve login for at få adgang til data og styring.
3. Kamera stream skal have en acceptabel forsinkelse så manuel styring via serveren er muligt.
4. Webserveren skal automatisk genstarte ved nedbrud.
5. API endpoints skal beskyttes mod uautoriseret adgang fra hackere.

4.2 Hardware- og softwarevalg

4.2.1 Hardwarevalg (Julian)

I starten af projektet besluttede vi os for at videreudvikle vores rover så den kunne mere end bare køre og samle objekter op og ned via en controller. Ifølge kravspecifikationen skulle vores rover bruge en ultrasonic sensor til at undgå sammenstød, bruge en DHT22 sensor til at måle luftfugtigheden og temperaturen samt bruge et kamera til at sende en direkte transmission til en server så roveren kunne blive brugt derpå.

DHT22 sensor

Til vores målinger af luftfugtighed og temperatur brugte vi en DHT22 sensor, grunden til vi valgte en DHT22 sensor fremfor andre sensorer var på baggrund af at den kunne måle lavere grader end nogle af de andre som stod til rådighed og det gør den til et optimalt valg da den skal bruges til at lave målinger af luftfugtigheden og temperaturen på månen for NASA. Sensoren sender digitale data til Arduinoen, målingen af data bliver sendt som 40 bits fordelt på 16-bit fugt, 16-bit temperatur og 8-bit checksum.²

L298N Motor drive module

Vi bruger en L298N Motor driver module som er en motorcontroller som bliver brugt til at styre DC-motorer fra en mikrocontroller som en Arduino Uno. L298N fungerer også som et mellemlid mellem Arduinoen og motoren da Arduinoen ikke kan levere nok strøm til motoren selv.

² Egelund Bloch, K., Klingenberg Vejs, N., Senker Tvede, R. & Johansson Rasmussen, J. (2026). Rover projekt – kort rapport. Zealand.

Komponenten indeholder to Dual H-Bridges, hvilket gør at man kan styre både retning og hastighed på op til to motorer samtidigt. En H-bridge er et elektronisk kredsløb, som gør det muligt at ændre strømretningen gennem en motor. Det betyder, at motoren kan køre både frem og tilbage. Ved at kombinere dette med PWM-signaler kan motorens hastighed også styres.² En ulempe ved L298N er at den ikke er særlig energieffektiv og bliver hurtig varm.

4x SG90 Servomotor

Der blev også gjort brug af 4 x servomotor – SG90. Servomotorerne er designet til at kunne dreje til specifikke vinkler, hvilket gør dem ideelle til en robotarm. I modsætning til DC-motorer kan servomotorer styres mere præcist. De har en drifspænding på 4,8-6V, drejevinkel på 0-180 grader, har en vægt på 9 gram og styres med en PWM-signaler fra en Arduino.

Den kører med 5V og har et drejningsmoment på 2,5 kg/cm. Motoren kan dreje fra 0 grader til 180 grader og dette styres via et PWM-signal på 50 Hz. Når signalet er 1ms er graden 0, ved 1,5ms er den 90 grader og ved 2ms er den 180 grader. Motoren har derudover tre ledninger, som har forskellige opgaver. Den brune ledning bruges til at give ground til systemet. Den røde ledning giver 5V til motoren. Den orange ledning bruges af Arduinoens mikrocontroller til at sende PWM-signaler til motoren.²

16x2 LCD-display modul

Et af de ældre komponenter som vi valgte med at fortsætte med at bruge til roveren, var 16x2 LCD-display modul som blev brugt til at vise information fra systemets sensorer og komponenter i realtid både på rover og controlleren. 16x2 LCD-display modul består af to linjer med plads til 16 tegn på hver linje. Derfor kan displayet vise op til 32 tegn i alt på displayet. Processen for at få data på vores LCD var som følgende, data bliver opsamlet via DHT 22 sensoren, data bliver sendt til Arduino Uno som viser data på controlleren og så sender Arduino Uno data til LCD, hvilket gør at den data der bliver opsamlet, kan blive vist på displayet på både roveren og controlleren. Et af de store fordele ved at vælge 16x2 LCD-display modul er at det giver brugeren en hurtig adgang til oplysninger uden nogen form for behov for ekstra udstyr.

10k ohm potentiometer

I vores projekt brugte vi et 10k ohm potentiometer som er en variabel modstand, som bliver brugt til at justere elektriske signaler igennem et kredsløb. Potentiometeret består af 3 terminaler og en drejeknap som gør det nemt at ændre modstandsværdien fra 0 til 10000 ohm. i forbindelse med 16x2 LDC-display modul bliver den oftest brugt til at justere displayets kontrast. Ved at dreje på knappen kan man justere LCD til enten at være mørkere eller lysere

NRF24L01

Et af de komponenter vi tog videre med fra det ældre projekt, var en Nrf24l01 som er et trådløst kommunikationsmodul der bruges til at sende og modtage data fx imellem to Arduinoer, robotter eller sensorer. Dette modul arbejder på 2,4 GHz-frekvensen som gør det muligt for den at fungere som både en sender og en modtager. I vores tilfælde bruger vi to Nrf24l01 den ene er tilsluttet på Arduinoen på kontrolleren og den anden er tilsluttet roveren.

Frekvensområdet er 2,4-2,525 GHz med 126 kanaler, og modulet understøtter datahastigheder på 250 kbps, 1 Nbps og 2 Mbps. Spændingen er 1,9-3,6 V og rækkevidden er op til ca. 100 meter afhængigt af antenne og omgivelser. Kommunikationen foregår via SPI. Fordelene er lavt strømforbrug, billig pris, nem integration med mikrocontrollere samt indbygget fejlhåndtering og automatisk genforsendelse. Ulemperne er begrænset rækkevidde uden ekstern antenne, følsom over for støj i 2,4 GHz-båndet og at den kræver stabil strømforsyning for optimal funktion.²

Dual axis joystick module

Dual axis joystick modulet bruges til at styre roveren. Det har to akser (X og Y), så man kan styre frem/tilbage og venstre/højre. Modulet bruger to potentiometre som ændrer den analoge spænding af joystickets position, som så aflæses af microcontrolleren. Joysticket har også en trykknop, som sender en digital værdi. Fordelene er at den er nem at bruge med microcontrollere, har en indbygget trykknop til ekstra funktioner og kræver få forbindelser. Ulemperne er at der kan forekomme stickdrift som skal tages højde for med deadzone, slid over tid og at den kan være følsom for små bevægelser og støj.²

Strømforsyningen

Vores strøm forsyning 2x 3,7v seriel (7,4v) i alt 6000mAH består af 2 batterier med 3,7V, hvor hver af dem har en kapacitet på 6000 mAH. Når serien bliver koblet til hinanden, bliver serien dobbelt så lang men kapaciteten forbliver den samme. Siden man skal kopple de to batterier så de giver 7,4V så det gør den til det mest perfekte batteri til alle ens komponenter på ens rover.

2x Mini DC gearmotor

Der bruges 1 motor til hvert hjul som er 2 x Mini DC gearmotor - 3-6Vdc / 200mA, det er en elektrisk motor med en gearkasse som øger trækraften og sænker hastigheden. Hver motor er regnet til at køre på mellem 3 til 6 volt og under en normal belastning bruger motoren 200 mA. T-motoren har et gear-forhold på 1:48. Det er en ret basal motor, da den har ingen indbygget encoder, hastighedsregulering eller positionsfeedback og kan ikke drives direkte fra en mikrocontroller.² Det er Arduino Uno som styrer motorerne og motorerne får strøm via enten en L298N eller en L293D, vores batteri på 7,4V forsyner hele systemet med strøm imens en buck converter sænker spændingen til de komponenter der skal have strøm.

Step down buck converter

For at sænke den spænding vi havde fra vores strømkilde til Raspberry Pi, og servo motorer brugte vi 2 x step down buck converter 4.5-24 dc-dc 3A 0.8-17V så vi kunne få et lavere niveau. I DC-input kan den få strøm fra 4,5V til 24V, hvor den leverer op til 3A (Ampere) og output kan justeres mellem 0,8V til 17V.

2x Arduino Uno

Arduino Uno er en mikrocontroller der bliver brugt til at styre elektronikprojekter såsom robotter sensorer og motorer. Arduino Uno kan programmeres til både at læse sensorer og til at styre

forskellige komponenter automatisk. Arduino Uno består af en mikrocontroller ATmega328P, 14 digitale pins, 6 analogt input, 5V drift spænding, USB-forbindelse til programmering og den kan køre direkte fra et batteri.

Den anbefalede inputspænding er 7-12 V, clockfrekvensen er 16 MHz og 6 af de 14 digitale ben understøtter PWM, som kan anvendes til at styre fx motorhastighed.²

Vi har valgt at beholde vores arduino som mikrokontroller til roveren da raspberry pi skal køre AI og video samt kommunikere med serveren. Det ville derfor kunne have effekt på realtids styring af roveren og ødelægge oplevelsen for bruger oplevelsen hvis vi kørte det hele over RaspberryPi. Så i stedet for at raspberryPi ser x antal komponenter den skal styre, ser den kun 1, arduinoen.

Videreudvikling af roveren

til videreudviklingen af roveren har vi tilføjet 4 komponenter.

- Raspberry Pi 3 model B
- Raspberry Pi 3 kamera
- HC-SR04 Ultrasonic sensor
- Step down buck converter

Raspberry Pi 3 Model B

For at kunne bruge Raspberry pi3 kameraet så skal den være tilsluttet et Raspberry pi3 modul B. Raspberry pi3 er på størrelse kreditkort og fungerer som en lille computer og bruges til mange forskellige ting så som programmering, serverer, sensorer osv. I vores tilfælde bruger vi Raspberry pi3 til en rover. Raspberry pi3 bruger en 1,2 GHz quad-core processor, har 1 GB ram, den har indbygget WI-FI og Bluetooth, 4 porte, et HDMI-port til en skærm, GPIO-pins til sensorer og elektronik, CSI-port til kamera og et microSD kort til styresystemet.

vi har valgt at udvide systemet med raspberryPi til kommunikationen mellem server og roveren. Da roveren skal kunne styres manuelt via serveren

Raspberry Pi 3 kamera

Vi brugte et Raspberry pi3 kamera til at få en direkte transmission fra roveren til vores server som gjorde det muligt at styre vores rover via serveren. Fordelen ved et Raspberry pi3 kamera er hurtigere dataoverførsel, har en kompakt størrelse, er bedre til roverprojekter og den er tilsluttet via en CSI-port som gør den væsentligt bedre da den også har lavere forsinkelser (latency), bedre til behandling af videoer og billeder, den er mere stabil til rover, robot og AI projekter og så bruger den mindre CPU-kraft.

HC-SR04 Ultrasonic sensor

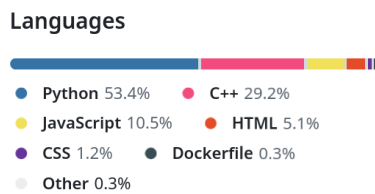
For at roveren skulle kunne køre/styre autonomt, tilsatte vi en ultrasonic sensor som sender lydbølger ud og registre, hvor lang tid det tager for lydbølgen at blive sendt tilbage, hvilket gør at den ikke støder ind i kanter og andre forskellige objekter på dens vej. Vi valgte en ultrasonic sensor HC-SR04 da ikke nok med at den er billig og nem at bruge så er den godt egnet til robotter og rovere da den kan måle fra 2 cm og helt op til 400 cm frem både i lys og i mørket. Der er dog nogle få ulemper ved HC-SR04 sensoren som er hvis overflader er bløde eller skrå så vil lyden ikke kunne reflekteres ordentligt og sensoren kan også blive påvirket af ekko eller støj som gør at den har svære ved at måle afstanden til objekter.

4.2.2 Softwarevalg (Rinus)

I vores projekt har vi anvendt en lang række diverse teknologier til at dække vores systems funktioner fra hardware til software. På Raspberry Pi er vores kodefiler skrevet i Python, fordi

officielt er Python standarden til enheden, og der findes et stort udvalg af biblioteker der støtter vores behov.

Til backend bruger vi FastAPI, da det er et hurtigt og moderne framework og derfor et godt valg til at bygge vores API endpoints og håndtere websocket kommunikation. Vores frontend er bygget op om HTML, CSS og JavaScript, som tilsammen udgør brugerfladen på webserveren.

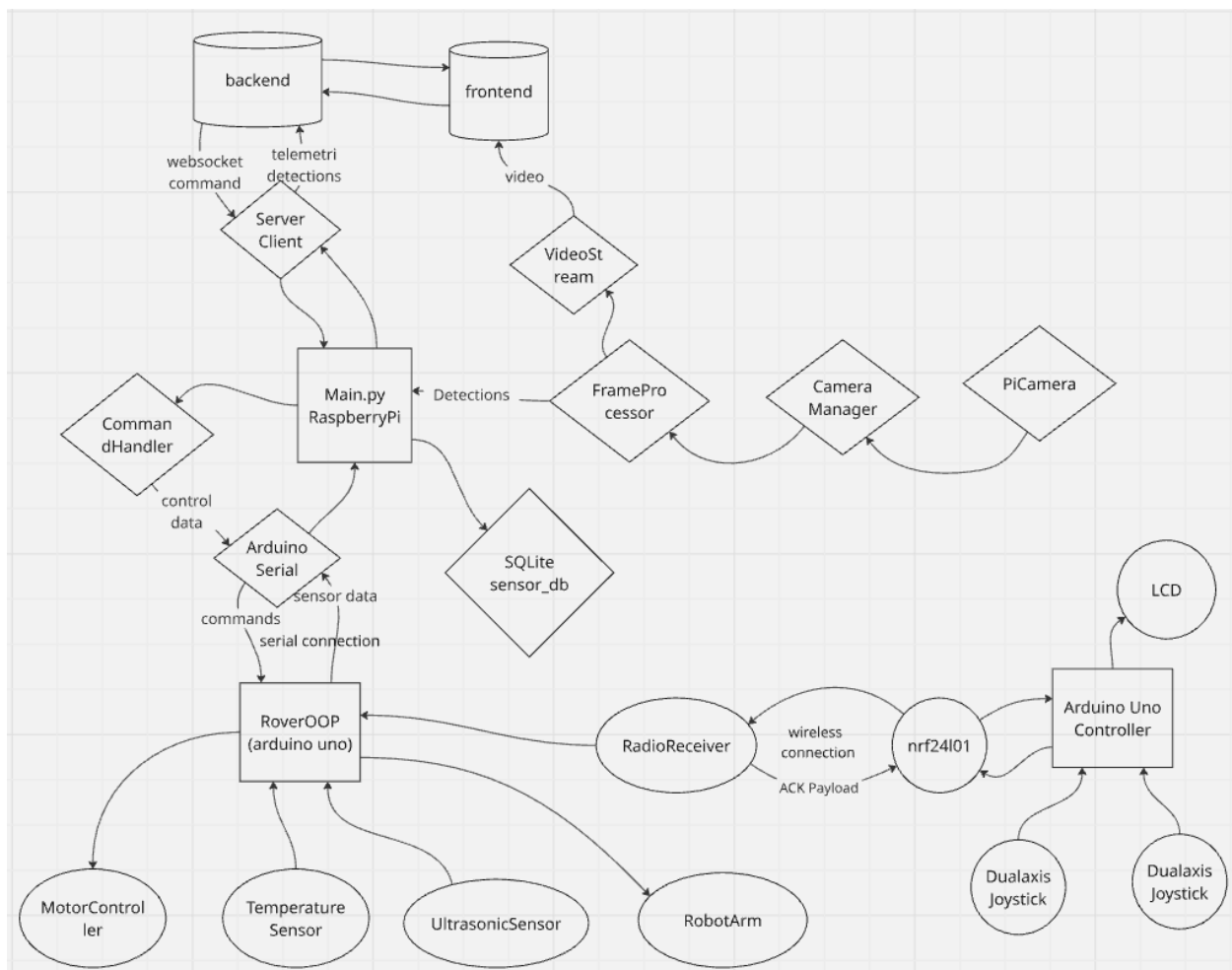


Til billedbehandling har vi brugt OpenCV, og til objektgenkendelse har vi gjort brug af TensorFlow Lite, da begge er velegende til at køre på en enhed med begrænsede ressourcer som vores Raspberry Pi.

Herudover gemmes vores sensordata og objekt detektioner i en MariaDB database, og forbindelsen til vores services sikres gennem en Nginx reverse proxy.

En mere uddybende begrundelse samt beskrivelse for hvert af de respektive valg af software dækkes i de efterfølgende afsnit i rapporten

4.3 Softwarearkitektur



Projektets software arkitektur blev udviklet med fokus på moduler, objektorienteret programmering og realtids kommunikation. Vi opdelte det i selvstændige software moduler så de enkelte dele kunne udvikles, testes og fejlfindes uafhængigt af hinanden.

Det gjorde det muligt at udvide det med nye funktioner uden at skulle omskrive hele programmet.

Arkitekturen blev fordelt mellem flere forskellige systemer og enheder herunder Arduinoen, RaspberryPi'en, backend-serveren, databasen og frontend. Kommunikation mellem systemerne blev håndteret gennem flere forskellige kommunikationslag. Der blev blandt andet anvendt NRF24L01 radiokommunikation til realtid styring af roveren, serialkommunikation mellem RaspberryPi og Arduino samt websocket kommunikation mellem rover og webapplikationen.

Den overordnede struktur kan deles op i:

- Arduino software
- RaspberryPi
- Backend
- Frontend
- Database system
- Kommunikationslag

4.3.1 Frontend (Kalle)

Vores frontend er et simpelt stack med HTML/CSS/JavaScript.

HTML og CSS står for selve det visuelle og layoutet på hjemmesiden, mens Javascript bruges til login og knapper, samt websocket og opdatering af data på hjemmesiden.

Frontend har to hovedfunktioner, lade brugeren logge ind og vise roverkontrolpanelet.

Vores kontrol panel har knapper, som bruges til at styre roveren med, når man holder en knap nede, laves der en JSON-kommando som sendes via websocket til vores backend. Vi har sørget for, at kommandoen sendes flere gange, når knappen holdes nede, for en mere stabil styring. Når knappen slippes, sendes der en stop kommando.

```
INFO: 172.21.0.3:47906 - "WebSocket /ws/control?token=<REDACTED>" [accepted]
Browser/operator connected to /ws/control
Control data received: {"type":"drive", "roverX":512, "roverY":0, "roverSW":false}
Control data received: {"type":"drive", "roverX":512, "roverY":0, "roverSW":false}
Control data received: {"type":"drive", "roverX":512, "roverY":0, "roverSW":false}
Control data received: {"type":"stop"}
```

Derudover viser vores hjemmeside også sensordata, som temperatur, luftfugtighed og afstand. Vores Raspberry står for, at servere denne data fra arduinoen, til vores backend via websocket. Her gemmes de i vores database. Frontend henter så de nyeste data fra backendens API og viser dem på hjemmesiden.

Vi bruger FastAPI til at bygge vores backend-API. Når hjemmesiden skal bruge ny data, sender JavaScript-koden et http-kald til vores backend med fetch. Backend modtager request og henter de specifikke data fra databasen og sender dem tilbage med JSON, som så vises på hjemmesiden. FastAPI står altså for at håndtere http-kald mellem front og backend.

Da vores kontrolpanel kan sende kommandoer til vores fysiske rover, er det vigtigt at alle ikke bare kan åbne siden og styre den. Derfor har vi en login page, hvor brugeren sender et username og et password til backend. Backend tjekker om det findes i databasen og returnerer en token hvis login er accepteret. Vi bruger tokens på to forskellige måder – vi har en bruger-token, som bruges til API-kald og kontrolsocket & så har vi en rover-token, som bruges som et bevis på, at det er vores rover som der forbinder til backend.

Authentication er altså ret vigtigt, da det sørger for at uvedkommende ikke kan få adgang. Bruger-token bruges altså som en form for midlertidig adgangsnøgle og bevis på, at brugeren har logget ind med korrekt password/username. Når API-kald laves, bruges denne token som et bevis på dette. Når der fx bliver sendt et GET på sensor data, sendes token med som tjekkes af backend, før data vises.

Rover-token ligger som en fast hemmelig token på Pi og backend. Når Pi opretter en websocket til backend, sendes denne token med og verificeres af backend. Vi har valgt at dele det op sådan, både for mere overskuelig struktur og sikkerhed.

```
INFO: 172.21.0.3:34302 - "GET /api/rover/status HTTP/1.1" 200 OK
INFO: 172.21.0.3:34312 - "WebSocket /ws/rover?token= " [accepted]
INFO: connection open
Rover connected to /ws/rover
```

På billedet ses først et GET-request til vores api, som returnerer 200 OK. Det er et status check, som sendes løbende. Det betyder at backendes status-endpoint svarer på frontends api-kald. Det betyder dog ikke nødvendigvis at roveren er online, da 200 OK kun viser at http-requested blev behandlet korrekt. Derefter ses websocket forbindelsen, hvor backend accepterer rover forbindelsen. Selve roverens online status afhænger af websocket forbindelsen til roveren og om backend har modtaget data for nyligt.

4.3.2 Backend (Kalle)

Backend er den del af systemet der binder frontend, database og rover sammen. Vores frontend snakker ikke direkte med databasen eller Arduino/Pi, men der bliver i stedet sendt requests til backend. Backend står så for at håndtere login, API-kald, Websocket og videresendelse af data. Det gør backend til et vigtigt mellemled i projektet. Når brugeren fx logger ind, henter sensordata eller sender en kommando fra kontrolpanelet, går det igennem backend først. Backend kan derfor både styre adgang, hente data fra databasen og sende kommandoer videre til Pi'en som derefter snakker videre med Arduinoen.

I vores backend bruger vi flere forskellige stykker software som hver især har en vigtig rolle i systemet. De vigtigste er FastAPI, Uvicorn, Docker, Nginx og databasen.

FastAPI bruges til backend-logikken, API-routes og webSocket-forbindelser.³ Uvicorn bruges til at køre FastAPI. Docker bruges til at køre backend i en container så det hele bliver mere kontrolleret. Nginx bruges som reverse proxy foran backend og sørger for at samle alt under samme path.

Docker

Vi bruger Docker til at køre vores backend i en container. Dette gør, at backend får sit eget isoleret miljø, hvor de rigtige dependencies og startkommandoer er defineret fra opstart.⁴

Det gør setuppet mere stabilt og nemmere at genstarte og genskabe, fordi alt ikke bliver manuelt installeret på serveren hver gang.

I vores projekt har vi en Dockerfile i backend, som beskriver hvordan backend-containeren skal bygges. Den bruger et Python image og installerer de dependencies vi har defineret i requirements.txt, kopierer backend-koden ind og starter appen med Uvicorn. Uvicorn er den server, som kører vores FastAPI app inde i containeren.

³ FastAPI Documentation. “WebSockets.”
<https://fastapi.tiangolo.com/advanced/websockets/>

⁴ Docker Documentation. “Docker overview.”
<https://docs.docker.com/get-started/docker-overview/>

Uvicorn er vigtigt fordi FastAPI ikke bare kører af sig selv. FastAPI er selve frameworket hvor vi definerer routes og endpoints, mens Uvicorn er den server der faktisk starter applikationen og tager imod HTTP og websocket trafik.⁵

Vi bruger også docker-compose til at starte backend. I docker-compose filen definerer vi blandt andet containerens navn, path, environment file, volumes og hvilken port backend skal bruge.⁶ Vores container hedder exam-backend og den kører på port 18765 inde i containeren, men brugeren tilgår ikke denne port direkte. I stedet går trafikken gennem Nginx på port 50325 som proxy'er videre til backend.

NAMES	IMAGE	STATUS	PORTS
nginx-proxy	nginx:latest	Up 4 days	0.0.0.0:50325->80/tcp, [::]:50325->80/tcp
exam-backend	examproject-backend	Up 4 days	18765/tcp

Billedet viser de kørende docker-containere. Backend-containeren kører internt, mens NGinx publicerer port 50325 udadtil.⁷

Vi bruger expose i stedet for bare at publicere backend porten direkte ud på serveren. Det betyder at backend porten kan bruges internt af docker og nginx, men at den ikke ligger som en

⁵ Uvicorn Documentation. "Uvicorn."
<https://www.uvicorn.org/>

⁶ Docker Documentation. "Docker Compose."
<https://docs.docker.com/compose/>

⁷ Docker Documentation. "Networking in Compose."
<https://docs.docker.com/compose/how-tos/networking/>

offentlig port på serveren.⁸

I vores compose starter vi Uvicorn med `-reload`, hvilket gør at backend automatisk genstarter hver gang der sker ændringer i koden. Det er smart under udvikling, fordi man hurtigt kan teste ændringer.

Docker gør også fejlfinding meget nemmere, fordi vi kan se præcis hvilke containere der kører med docker ps, og vi kan læse docker logs, det har været vigtigt under udviklingen af projektet, fordi vi kunne se om API-kald kørte korrekt samt om websocket-forbindelser blev accepteret og kommandoerne fra kontrolpanelet faktisk blev modtaget. Docker løser dog ikke alt automatisk, man skal stadig forstå hvordan ports. Miljøvariabler og netværk hænger sammen. Hvis backend-containeren kører, men nginx peger på en forkert port eller en forkert container, virker hjemmesiden ikke. Derved er docker altså ikke bare et magisk deployment værktøj der virker helt af sig selv, men et værktøj der giver mere kontrol og overview.

Samlet set bruger vi Docker for at gøre backend mere isoleret, ensartet og lettere at starte og fejlfinde. Backend bliver pakket ind i en container med de dependencies den skal bruge, mens nginx kan sende trafik videre til containeren internt. Det giver en overskuelig struktur.

Nginx

Vi bruger Nginx som reverse proxy foran vores backend. Det betyder at brugeren ikke rammer backend-containeren direkte, men i stedet rammer Nginx, som sender trafikken videre til interne

⁸ Docker Documentation. "Compose file reference: expose."
<https://docs.docker.com/reference/compose-file/services/#expose>

services.⁹ Det giver en kontrolleret struktur, fordi vi ikke behøver at åbne flere porte end højst nødvendig.

I vores setup kører backend på port 18765 inde i docker-miljøet, mens Nginx samler alt på port 50325 udadtil. Brugeren går derfor ikke direkte ind på backendens port. I stedet bruger vi routes som starter med /rover/ - når browseren fx kalder /rover/api/sensor-data, så er det Nginx der først modtager requestet og sender det videre til backends interne API-route.

```
proxy_pass http://grafana:3000/grafana/;
location /rover/static/ {
    proxy_pass http://exam-backend:18765/static/;
location /rover/ws/ {
    proxy_pass http://exam-backend:18765/ws/;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";
location /rover/stream.mjpg {
    proxy_pass http://host.docker.internal:18000/stream.mjpg;
location /rover/ {
    proxy_pass http://exam-backend:18765/;
```

Udsnit af Nginx-konfigurationen hvor bl.a /rover/, /rover/ws liver sendt videre til interne services.

Her ses også websocket-headerne Upgrade og Connection, som er nødvendige for at websocket-forbindelsen kan holdes åben.

Det betyder altså at Nginx fungerer som en slags mellemstation mellem browseren og backend. Browseren behøver ikke kende backendens interne port, container navne eller Docker-netværk, den skal bare kalde den route, som er indstillet via Nginx. Det gør systemet meget mere

⁹NGINX Documentation. "NGINX Reverse Proxy.
<https://docs.nginx.com/nginx/admin-guide/web-server/reverse-proxy/>

ryddeligt, fordi frontend, API, Websocket & stream samles under samme path.

En vigtig del er Websocket. Almindelige http-kald fungerer som request og response, men websocket er en åben forbindelse som skal holdes i live.¹⁰ Derfor skal Nginx konfigureres korrekt til websocket, ellers kan frontend godt loades siden, men selve styringen af roveren og live stream fejler. Nginx sender blandt andet Upgrade og Connection headers videre, så forbindelsen bliver opgraderet fra almindelig http til websocket.¹¹

En af fordelene ved Nginx, er at vi kan skjule den interne struktur. Udefra ser det ud som om hele systemet ligger under /rover/, men internt kan det være flere forskellige services og porte. Det er bedre end at brugeren skal tilgå backend på en port, live feed på en anden port og evt andre services på en tredje.

Det har også en sikkerhedsmæssig fordel. Hvis backend ikke behøver være direkte åben som en offentlig port, kan vi lade Nginx være det lag der bestemmer hvad der bliver eksponeret.

Backend kan ligge mere internt, mens nginx kun sender trafik videre til de routes vi rent faktisk ønsker at bruge. Det betyder ikke at Nginx automatisk gør alt sikkert, men det giver et bedre og mere kontrolleret setup.

Nginx har også været en del af fejlfindingen, når vi fx fik 502, betød det ikke nødvendigvis at Nginx selv var problemet. Det betød ofte at Nginx prøvede at sende trafik videre til en service, som ikke svarede korrekt. Det kunne være backend-containeren, live feed streamen eller en forkert port. Derfor var Nginx et vigtigt punkt i fejlfinden, fordi det står imellem brugeren og

¹⁰ MDN Web Docs. "The WebSocket API."
https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API

¹¹ NGINX Documentation. "WebSocket proxying."
<https://nginx.org/en/docs/http/websocket.html>

resten af systemet.

Samlet set bruger vi Nginx til at samle adgangen til vores system, skjule interne ports og sende trafik videre til backend. Det gør deployment mere realistisk, fordi brugeren ikke bare tilgår en tilfældig backend-port direkte, men går gennem en reverse proxy, som styrer trafikken ind til systemet.

API og vores API-routes

Vi bruger API'et som den normale kommunikation mellem front og backend. Det betyder at når hjemmesiden skal hente data, så sender JavaScript koden et http-kald til backend, typisk med fetch. Backend modtager requestet, behandler data og sender et svar tilbage som JSON. Frontend bruger så det json svar til at opdaterer det der vises på hjemmesiden.

Et eksempel er sensordata. Når frontend skal vise temperatur, luftfugtighed og afstand, kalder den backendens API. Backend henter så de nyeste værdier fra databasen og returnerer dem til frontend som JSON. Frontend snakker derfor ikke direkte med databasen, men kun med backendens API. Det gør systemet mere sikkert og overskueligt, fordi backend bestemmer hvilke data frontend må få adgang til.

Vi har blandt andet API-routes til login, rover status, sensordata og detections. Login-route bruges når brugeren sender username og password til backend. Hvis login er korrekt, returnerer backend en token som kan bruges senere ved API-kald. Status-routen bruges til at tjekke, om backend vurderer roveren som online. Sensor-data-routen bruges til at hente de nyeste målinger

fra databasen. Detection-route bruges til at hente de nyeste resultater fra PI'ens kamera-del.

Nogle af de vigtiges routes vi bruger:

POST /login

- Bruges til login. Frontenden sender username og password til backend. Hvis de er korrekte, returneres en token.

GET /api/rover/status

- Bruges til at tjekke roverens status. Frontenden kan kalde denne route løbende for at se, om backend vurderer roveren som online.

GET /api/sensor-data

- Bruges til at hente sensordata, backend henter data fra databasen og returnerer JSON.

GET /api/detections

- Henter detection-data som Pi'en har sendt til backend.

```
@app.get("/")
@app.get("/api/rover/status")
@app.get("/api/sensor-data")
@app.get("/api/detections")
@app.post("/register")
@app.post("/login")

@app.websocket("/ws/control")
@app.websocket("/ws/rover")
```

Udsnit fra backend-koden, hvor vores FastAPI-routes er defineret. Her ses de almindelige routes til frontend som blandt andet bruges til login, rover-status, sensordata og detections. Nederst ses også websocket-routes til kontrolpanelet og roveren.

En vigtig detalje er at browseren ikke nkaller de interne backend-routes direkte. Fordi vi bruger nginx som reverse proxy, kalder frontenden routes med /rover/ foran. For eksempel kan frontend kalde /rover/api/sensor-data, mens Nginx sender requested videre til backendens interne route /api/sensor-data. På den måde samles hele webdelen under samme /rover/ path.

API'et er også koblet sammen med authentication. Når brugeren er logget ind, sender frontenden token med ved API-kald, så backend kan tjekke om brugeren har adgang. Det betyder, at backend ikke bare returnerer data til hvem som helst. Før data sendes, skal token valideres.

I det store hele bruger vi API til den strukturerede dataudveksling mellem frontend og backend. Frontend spørger backend om data, backend behandler data og sender tilbage som JSON. Det gør det lettere at holde frontend, backend og database adskilt, og giver backend rollen som det centrale punkt for data og adgang.

Sikkerhed

I løbet af vores projekt, har sikkerhed været en vigtig del, fordi systemet ikke bare er en almindelig hjemmeside. Hjemmesiden giver adgang til at styre en fysisk rover, se live video og hente data. Det betyder at et sikkerhedsproblem ikke kun ville være, at nogen kunne se en side de ikke burde se. I værste fald kunne en uvedkommende sende kommandoer videre til roveren og kigge med på roverens kamera live-feed. Derfor har vi tænkt over, hvilke dele af systemet der er åbne og hvilke dele der skal være skjult samt teste at det faktisk fungerer sådan.

En af de vigtigste ting i vores opsætning er at begrænse angrebsfladen. Med angrebsflade menes porte, services og endpoints, som kan rammes udefra. Hvis alt bare ligger åbent, er der mange steder noget kan gå galt, derfor har vores mål været, at kun det nødvendige skal være tilgængeligt. Backend, database, grafana og interne porte bør ikke ligge frit åbent til internettet.

De skal enten ligge bag Nginx, kun være tilgængelige lokalt på serveren, eller tilgås gennem en SSH-tunnel.

Serveradgangen via SSH er nok et af de vigtigste sikkerhedspunkter. Hvis nogen får SSH-adgang til serveren, vil de i praksis have adgang til alt. De vil kunne stoppe docker-containere, ændre Nginx settings, læse filer og ændre kode. SSH har også været en praktisk del af vores udvikling, fordi vi har brugt SSH tunnels. Det betyder at vi kan tilgå interne services fra vores egen maskine, uden at åbne dem direkte mod internettet.

UFW og Fail2Ban

Som en del af sikkerheden har vi også sat firewall og Fail2ban op. Målet med dette er ret simpelt at lukke alt der ikke skal bruges, holde øje med den adgang der er og sikre os imod brute-force loginforsøg.

UFW står aktivt på serveren og er sat op med den incoming, allow outgoing og deny routed.¹² Det betyder i praksis at serveren som udgangspunkt ikke accepterer indgående trafik medmindre vi specifikt har åbnet for det. Det er en vigtig del af vores setup fordi der kører flere forskellige services på serveren, men det er ikke meningen at alt skal kunne rammes udefra.

De vigtigste åbne porte er SSH på port 22 og nginx på port 50325. SSH er nødvendigt for at vi kan tilgå serveren. Port 50325 bruges til reverse proxy, altså den port hvor webdelen kan tilgås gennem Nginx. Backend-porten 18765 er altså ikke publiceret direkte ud gennem Docker, men ligger kun eksponeret internt som en container-port.

Vi har også sat UFW til, at vores MariaDB port 3306 kun er tilladt fra docker-netværket, da

¹² Ubuntu Server Documentation. "Firewall."
<https://ubuntu.com/server/docs/how-to/security/firewalls/>

databasen kun bruges af backend og interne services.

Der er også en regel for port 18000, den port hænger sammen med vores stream/live-feed, hvor der har været noget mapping omkring MJPEG-stream, igen er pointen at den ikke bare er åbnet frit til alle, men er begrænset til det interne netværksområde.

Fail2ban er også aktivt på serveren. Det kører som en service og har aktiv sshd jail, som er sat op til at beskytte SSH adgang, hvilket giver mening fordi SSH er den vigtigste direkte adgang til serveren.¹³ Hvis nogen prøver at bruge force login, kan Fail2ban reagere på gentagne fejl på login og banne IP'en midlertidigt. Vores Fail2ban er sat til maxretry = 3, findtime = 10min, bantime = 1h. Det betyder at hvis en IP fejler login for mange gange inden for 10 minutter, kan den blive blokeret i en time. Det fjerner en del simple brute force forsøg og gør SSH mindre åben for misbrug.

Grafana og observability

Vi har også brugt Grafana som en del af vores sikkerhed og driftsmæssige overvågning. Grafana er ikke som sådan sikkerhed i sig selv, men det giver os et overblik over hvad der sker på serveren.¹⁴ Det har været vigtigt i vores projekt fordi serveren kører flere forskellige services.

Da vores hjemmeside ikke har stået permanent offentligt deployed på internettet, har den vigtigste angrebsvinkel mod serveren primært været SSH. Derfor har vi haft fokus på at logge SSH-aktivitet i Grafana, hvor vi har lavet et panel der viser ssh sessions fra /var/log/auth.log,

¹³ Fail2Ban GitHub. "Fail2Ban: ban hosts that cause multiple authentication errors."
<https://github.com/fail2ban/fail2ban>

¹⁴ Grafana Documentation. "Dashboards."
<https://grafana.com/docs/grafana/latest/visualizations/dashboards/>

hvor vi kan se hvornår SSH-sessions bliver åbnet eller lukket og hvilken bruger der logger ind. Det gør det lettere hurtigt at opdage hvis der sker login som ikke passer med vores arbejde.

Vi har også lavet et panel til "High risk commands", som filtrerer sudo-kommandoer fra auth.log og viser kommandoer der potentielt kan ændret systemet. Det er for eksempel kommandoer som systemctl, ufw, pkill, chmod, chown, curl, wget, ssh. Et eksempel er at man i dashboardet kan se kommandoer som stop af cloudflared, ufw status og pkill. Det passer med vores udviklingsproces, hvor vi testede cloudflare tunnel, men stoppede den igen bagefter så den ikke stod unødvendigt åbent.

Derudover har vi også et docker-status panel, hvor vi kan se hvilke containere der bliver kørt og om de er oppe.

4.3.3 PiCore (Nicklas)

RaspberryPi fungerer som et bindeled mellem roverens hardware, kamera. AI funktioner og serverens backend. På RaspberryPi blev der udviklet en software del kaldet PiCore. Den havde ansvar for de opgaver der krævede mere processorkraft eller netværkskommunikation end Arduinoen kunne håndtere.

Hvor arduinoen primært stod for hardware styring, stod PiCore for de mere avancerede funktioner som kamera streaming, Ai objekt/farve genkendelse, serial kommunikation, og websocket kommunikation med serveren.

PiCore bestod af følgende overordnede dele.

- kamera og videostreaming
- AI og billedbehandling
- serial-kommunikation med Arduino

- websocket klient til backend serveren
- sensorhåndtering
- detection-data til databasen

Hovedprogrammet `main.py` fungerede som det centrale samlingspunkt i PiCore. Her blev de forskellige moduler startet og koordineret. Programmet initialiserede blandt andet kameraet, oprettede forbindelse til Arduinoen via serial, startede WebSocket forbindelsen til backend serveren og kørte det løbende program loop.

Kameraet blev håndteret gennem `CameraManager`, som stod for at hente frames fra Raspberry Pi-kameraet. Disse frames blev sendt videre til `FrameProcessor`, hvor billedet blev analyseret med OpenCV og TensorFlow Lite. Her blev object detection og farvegenkendelse udført, og resultatet blev brugt til både at markerer detections i video og sende detection-data videre til backend-serveren.

Den behandlede video stream blev håndteret af `VideoStream`, som gjorde det muligt at vise videofeedet i webapplikationen. Videostreamen blev holdt adskilt fra websocket kommunikationen, da video kræver langt mere båndbredde end simpel kontrol og sensordata. Dette gjorde systemet mere stabilt da videostreamen ikke belastede realtid styring af roveren og sensordata.

Kommunikationen mellem RaspberryPi og arduino skete igennem `ArduinoSerial`. Dette modul læste sensordata fra arduinoen og sendte kommandoer tilbage til arduinoen. Sensordata blev sendt fra arduinoen som tekstbeskeder og blev behandlet og sendt videre af PiCore til backend.

WebSocket kommunikationen mellem RaspberryPi og serveren blev håndteret af `ServerClient`. Dette modul sendte sensordata, detections og rover status til serveren. Samtidig modtog det kommandoer fra webapplikationen som derefter blev sendt videre igennem `CommandHandler`

Denne opdeling gjorde RaspberryPi til et kommunikation- og behandlingslag imellem det lave hardware niveau på Arduinoen og websystemet på serveren. PiCore gjorde det derfor muligt at kombinere kamera, AI, sensordata, serial-kommunikation og WebSocket-kommunikation i en samlet softwaredel.

4.3.4 Arduino firmware (Nicklas)

Arduino'en stod for den direkte styring af sensorer og motorer og blev udviklet igennem OOP struktur og blev opdelt i flere klasser.

- Motorcontroller
- TemperatureSensor
- UltralydSensor
- RadioReceiver
- RobotArm

Det gjorde systemet mere overskueligt og lettere at udvide med nye funktioner under udviklingen.

Arduionens havde desuden RadioReceiver som modtog Kontrol packets (kommandoer) fra en controller og sendte ack packets tilbage med sensordata.

Arduinoen håndterede bl.a.:

- servoer
- sensorer
- radiokommunikation
- autonomous mode
- obstacle avoidance
- motorstyring

Sensordata blev sendt videre til RaspberryPi igennem serial kommunikation.

4.4 Kommunikation

4.4.1 NRF24L01 radio (Nicklas)

En af mulighederne for at styre roveren er via controlleren. Her anvendes to NRF24L01 moduler.

Receiveren som er placeret på roveren og transmitteren der er placeret på Controlleren.

Kommunikationen sker igennem en 2,4 GHz forbindelse, hvilket gør at roveren kan styres trådløst uden en netværksforbindelse.

Controlleren sender kontrol pakker med joystick positioner og knaptryk. Denne data bliver modtaget af receiveren, der sender det videre til arduino, som bruger dataen til at styre motorer og robotarmen.

```
struct ControlPacket {  
    int roverX;  
    int roverY;  
    bool roverSW;  
  
    int armX;  
    int armY;  
    bool armSW;  
};  
  
struct SensorPacket {  
    float temperatureC;  
    float humidity;  
};
```

Roveren gør brug af NRF24L01's AckPayload funktion. Når roveren modtager en kontrol pakke fra controlleren, sender roveren en bekræftelse om at pakken er modtaget. Denne bekræftelse indeholder sensordata herunder temperatur og luftfugtighed som herefter bliver vist på LCD'en, på controlleren.

```
radio.writeAckPayload(0, &sensorData, sizeof(sensorData));
```

Kommunikationen løber derfor i begge retninger selvom den ene er receiver og den anden transmitter. Dette giver brugeren mulighed for at styre roveren og samtidig observere sensordata.

For at undgå ukontrolleret kørsel overvåges forbindelsen. Hvis der ikke modtages kontrolpakker inden for en bestemt tidsperiode, aktiveres en timeout funktion, som stopper roveren. Dette sørger for, at roveren ikke fortsætter med at køre ved tab af forbindelsen.

```
if (millis() - lastReceiveTime > SIGNAL_TIMEOUT) {  
    | motors.stop();
```

4.4.3 WebSocket (Nicklas)

Til kommunikation mellem Roveren, frontend og backend valgte vi websocket. Det blev valgt fordi projektet krævede realtid kommunikation med lav forsinkelse mellem flere dele af systemet. Dette var især vigtigt i forbindelse med fjernstyring af roveren.

I modsætning til almindelig HTTP kommunikation oprettet websocket en vedvarende forbindelse imellem klient og server. Ved almindelig HTTP request skal klient hele tiden sende nye requests til serveren. Med websocket kan data sendes løbende i begge retninger i en konstant forbindelse. Det reducerer forsinkelser og gør systemet mere responsivt.

I projektet blev websocket anvendt mellem RaspberryPi og backend og frontend. Kommunikationen var derfor opdelt i flere lag hvor backend agerede som mellemlid mellem rover og frontend applikationen.

På RaspberryPi blev ServerClient anvendt som websocket klient og stod for at oprette forbindelse til backend og håndtere udveksling af data.

Ved opstart oprettes forbindelse til serveren:

```
def connect(self):
    self.running = True

    self.thread = threading.Thread(
        target=self._connection_loop,
        daemon=True
    )

    self.thread.start()
```

Når RaspberryPi starter, oprettes websocket klienten gennem connect() funktionen. Forbindelsen køres i en baggrundstråd ved hjælp af pythons threading bibliotek. Det gør det muligt at opretholde kommunikationen med backend uden at blokere resten af programmet. Kamera, AI behandling, serial kommunikation og websocket kommunikation kan derfor køre samtidigt.

Når RaspberryPi modtager sensordata fra Arduinoen gennem serial kommunikation, bliver dataen sendt videre til backend-serveren som JSON-data:

```
json_string = json.dumps(data)
```

Ved at bruge json format kan forskellige typer data fx temeperatur, afstand og objekter sendes igennem samme forbindelse

et eksempel på data som blev sendt til backend serveren kunne være:

```
return self.send({
    "type": "sensor_data",
    "temperature": temperature,
    "humidity": humidity,
    "distance": distance,
    "timestamp": time.time()
})
```

Objekt detections blev ligeledes sendt igennem websocket forbindelsen til backend og havde et objekt navn, confidence score, og farve.

Frontend kommunikerede med backend via websocket forbindelser det gjorde det muligt at opdatere applikationen uden at skulle genindlæse siden.

Når backend modtog nye sensordata eller detections fra roveren blev de straks sendt videre til frontend applikationen.

Applikationen viste bl.a.

- Temperatur
- Luftfugtighed
- Afstand
- Farve
- Objekter

Frontend kunne samtidig sende kommandoer tilbage til backend, som sendte det videre til raspberryPi, der sendte dem igennem ArduinoSerial til Arduinoen.

Brugeren kunne fx. aktivere autonomous mode, styre armen eller køre rundt med roveren.

Realtids kommunikation var en central del af projektet da roverens styring krævede hurtig respons mellem rover og bruger. Hvis kommunikationen havde for høje forsinkelser ville manuel styring blive vanskelig og upræcis.

Websocket gjorde det muligt at holde konstant forbindelse mellem systemerne, hvilket reducerede forsinkelser sammenlignet med HTTP requests.

Videostreamen blev holdt adskilt fra websocket kommunikationen. Videofeedet blev sendt gennem en separat MJPEG stream ved hjælp af VideoStream, mens websocket forbindelsen udelukkende blev anvendt til sensordata, detections og kontrolkommandoer.

Systemet kommunikation kunne derfor opdeles således:

websocket

Rover < > backend < > frontend

MJPEG

Pi camera >> frontend

Denne opdeling reducerede belastningen på systemet og gjorde det mere stabilt.

Brugen af websocket gjorde det muligt at opbygge et realtidssystem, hvor sensordata, objekt detections og kommandoer kunne udveksles hurtigt mellem rover, backend og frontend. Den vedvarende forbindelse gjorde systemet mere responsivt og forbedrede brugeroplevelsen under fjernstyring af roveren. Opdelingen mellem videostreaming og websocketdata gjorde samtidig systemet mere stabilt og reducerede belastningen på kommunikationen.¹⁵

¹⁵ Mozilla Developer Network (MDN), "WebSocket API (WebSockets)"
https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API

https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API

CommandHandler (Nicklas)

Commandhandler fungerer som et bindeled mellem websocket kommunikationen og serial kommunikationen med arduino. Når RaspberryPi modtager en JSON besked fra backend, sendes den videre til CommandHandler, som identificerer hvilken kommando det er. Herefter værdierne bliver sat ind i en kontrol pakke. Herefter bliver kontrolpakken sendt videre til ArduinoSerial der omskriver det til Arduino format og sender det videre til Arduino.

4.4.4 Seriel forbindelse mellem Raspberry Pi og Arduino (Rinus)

Kommunikationen mellem Raspberry Pi og Arduino foregår via pyserial biblioteket over USB serielt i to retninger.¹⁶ Først sendes der control kommandoer fra Pi'en via arduino_serial.py til Arduino'en, og derefter sendes der sensordata tilbage fra Arduino'en til Pi'en via RoverOOP.ino. Pi'en sender bevægelses packets til Arduino'en i fast tekstformat CONTROL: roverX, roverY, roverSW, armX, armY, armSW.

```
79     try:
80         message = (
81             f"CONTROL:"
82             f"{packet['roverX']},"
83             f"{packet['roverY']},"
84             f"{int(packet['roverSW'])},"
85             f"{packet['armX']},"
86             f"{packet['armY']},"
87             f"{int(packet['armSW'])}\n"
88         )
89
90         with self.lock:
91             self.ser.write(message.encode())
```

Arduino'en modtager disse gennem readSerialControl(), hvor den har læst indtil linjen \n, den fjerner CONTROL præfikset og parser de 6 separate værdier ud i et array med int values[6].

¹⁶ https://pyserial.readthedocs.io/en/latest/pyserial_api.html

```

234 void readSerialControl() {
235     if (Serial.available()) {
236         String line = Serial.readStringUntil('\n');
237
238         if (line.startsWith("CONTROL:")) {
239             line.remove(0, 8);
240
241             int values[6];
242             int index = 0;
243
244             while (line.length() > 0 && index < 6) {
245                 int commaIndex = line.indexOf(',');
246
247                 if (commaIndex == -1) {
248                     values[index++] = line.toInt();
249                     break;
250                 }
251
252                 values[index++] = line.substring(0, commaIndex).toInt();
253                 line.remove(0, commaIndex + 1);
254             }

```

Arduino'en sender sensordata tilbage til Pi'en hvert 2. sekund i formatet SENSOR:TEMP, SENSOR:HUM, SENSOR:DIST.

```

107 static unsigned long lastPrint = 0;
108
109 if (millis() - lastPrint > 2000) {
110     lastPrint = millis();
111
112     Serial.print("SENSOR:TEMP:");
113     Serial.println(sensorData.temperatureC);
114
115     Serial.print("SENSOR:HUM:");
116     Serial.println(sensorData.humidity);
117
118     Serial.print("SENSOR:DIST:");
119     Serial.println(ultrasonic.getDistanceCm());
120 }

```

Pi'en læser disse i baggrunden med readloop(), som konstant lytter på serial porten og videregiver hver linje til parse_arduino_message().

```

101     def read_loop(self):
102         print("Serial read loop started.")
103
104         while self.is_running:
105
106             try:
107                 if self.ser.in_waiting > 0:
108                     raw = self.ser.readline()
109                     line = raw.decode("utf-8").strip()
110
111                     if line: # ignore empty lines
112                         self.parse_arduino_message(line)

```

parse_arduino_message() splitter beskeden på : og gemmer værdien i sensor_data.

```

126     def parse_arduino_message(self, line):
127         parts = line.split(":")
128
129         # check it's a sensor message
130         if len(parts) >= 3 and parts[0] == "SENSOR":
131             key = parts[1]
132             val = parts[2]
133             self.sensor_data[key] = val
134         else:
135             print(f"Arduino: {line}")
136

```

På den måde kan get_sensor() til ethvert tidspunkt slå en værdi op ved navn, og metoderne get_distance_cm(), get_temperature(), get_humidity() bruge den til at returnere de enkelte sensorværdier.

```

137     # get a sensor value by name
138     def get_sensor(self, key):
139
140         if key in self.sensor_data:
141             return self.sensor_data[key]
142
143         return None # not received yet
144
145     def get_distance_cm(self):
146         val = self.get_sensor("DIST")
147

```

4.5 Computer Vision og AI

4.5.1 OpenCV (Rinus)

Vi har brugt OpenCV til at gøre meste af det tunge arbejde inden for billedbehandling i vores Raspberry Pi. OpenCV er et open source-bibliotek til computer vision, som gør det muligt at behandle og analysere billeder fra vores kamera i realtid. Biblioteket er opbygget i moduler, og vi har primært anvendt Image Processing modulet, som havde en række underkategorier i modulet som vi gjorde brug af.¹⁷ I `color_detection.py` brugte vi Color Space Conversions til at konvertere farverummet fra RGB til HSV for at få de rigtige farver på billedet (det var blåligt før konverteringen).

```
26 def detect(frame_rgb, colors=None):
27     # converts an image from one color space to another
28     hsv = cv.cvtColor(frame_rgb, cv.COLOR_RGB2HSV)
29
```

Image Filtering brugte vi til at slørre billedet for at reducere støj inden farveanalysen. Herefter brugte vi `cv.inRange` fra core modulet til at skabe en binær maske, hvor pixels inden for et farveinterval farves hvidt, og resten farves sort som gør det muligt at isolere en specifik farve ift. farvedetektionen.

```
37     mask = np.zeros(hsv.shape[:2], dtype=np.uint8)
38     for (lower, upper) in COLOR_RANGES[color]:
39         # checks if array elements lie between the elements of two other arrays
40         mask |= cv.inRange(hsv, lower, upper)
41
42     # findContours retrieves contours from the binary image
43     # CHAIN_APPROX_SIMPLE removes all redundant points and compresses
44     # the contour, thereby saving memory
45     contours, _ = cv.findContours(mask, cv.RETR_EXTERNAL, cv.CHAIN_APPROX_SIMPLE)
46
```

Derudover anvendte vi Structural Analysis and Shape Descriptors til at detektere samt lokalisere farvede objekter i billedet.

¹⁷ <https://docs.opencv.org/4.x/d1/dfb/intro.html>

```

45     contours, _ = cv.findContours(mask, cv.RETR_EXTERNAL, cv.CHAIN_APPROX_SIMPLE)
46
47     for cnt in contours:
48         area = cv.contourArea(cnt)
49         if area < MIN_CONTOUR_AREA:
50             continue
51
52         # (x, y) represents the top-left corner and (w, h) width and height dimensions
53         x, y, w, h = cv.boundingRect(cnt)
54
55         M = cv.moments(cnt)
56         if M["m00"] == 0:
57             continue
58         cx = int(M["m10"] / M["m00"])
59         cy = int(M["m01"] / M["m00"])

```

Vi bruger `cv.findContours` til at finde konturer i den binære maske, `cv.contourArea` til at filtrere støj fra ved at ignorere konturer under en minimumstørrelse på 500, `cv.boundingRect` til at lokalisere objektet med en afgrænsningsboks og `cv.moments` til at beregne objektets centroid.¹⁸

```

22
23     MIN_CONTOUR_AREA = 500
24
25

```

En kontur i sig selv er en kurve der forbinder alle punkter langs kanten af et område med samme farve. Det er en effektiv måde på at præcist genkende farvet objekter i et billede.

I `frame_processor.py` anvendte vi `Drawing Functions` fra det samme `Image Processing OpenCV` modul til at annotere de behandlede frames med de detektede objekter. `Cv.rectangle` tegner en boks rundt om objektet, og `cv.putText` skriver en tekstlabel med navnet på farven direkte på billedet.

¹⁸ https://docs.opencv.org/4.x/d4/d73/tutorial_py_contours_begin.html

```

26         if self.detect_colors is not None:
27             results["colors"] = color_detection.detect(blurred, self.detect_colors)
28             for obj in results["colors"]:
29                 x, y, w, h = obj["bbox"]
30                 # Draw bounding_box
31                 cv.rectangle(annotated, (x, y), (x + w, y + h), _TEXT_COLOR, 3)
32                 # Draw label and score
33                 cv.putText(annotated, obj["color"],
34                           (_MARGIN + x, _MARGIN + _ROW_SIZE + y),
35                           cv.FONT_HERSHEY_PLAIN, _FONT_SIZE, _TEXT_COLOR, _FONT_THICKNESS)

```

Det samme gælder for TensorFlow Lite detektionen, hvor vi bruger cv.putText igen til at skrive objektnavnet og konfidens scoren på billedet.

```

42         cv.rectangle(annotated, (x, y), (x + w, y + h), _TEXT_COLOR, 3)
43         # Draw label and score
44         result_text = obj["label"] + ' (' + str(obj["score"]) + ')'
45         cv.putText(annotated, result_text,

```

4.5.2 Objektdetekion (Rinus)

Udover OpenCV har vi anvendt en TensorFlow Lite model til at detekte objekter i vores kamerastream. TFLite er som navnet antyder et light weight deep learning framework, der var udviklet af Google. Grunden til vi valgte lige netop det er at TF Lite er optimeret til at køre på

enheder som vores Raspberry Pi med begrænsede ressourcer, uden at gå på kompromis med kvaliteten af detektionerne.¹⁹

Den model variation vi bruger kaldes for EfficientDet-Lite0 Object detection model, som var trænet på COCO 2017 datasættet, der indeholder mere end 200.000 billeder og 80 objekt kategorier.²⁰

Selve modellen er en .tflite fil der er gemt lokalt på Pi'en. Den indlæses via MODEL_PATH ved opstart.

```
21 MODEL_PATH = os.path.join(os.path.dirname(__file__), "models", "1.tflite")
22
```

Når en frame kommer fra kameraet, konverteres det til et TensorImage objekt og sendes direkte ind i efficientdet modellen, som returnerer en liste af detekterede objekter med label, en konfidens score og afgrænsningsboks, der markerer hvor i billedet objektet befinder sig.

```
36     def detect(self, frame_rgb):
37         # Create a TensorImage object from the RGB image.
38         input_tensor = vision.TensorImage.create_from_array(frame_rgb)
39
40         # Run object detection estimation using the model.
41         detection_result = self.detector.detect(input_tensor)
42
```

Vores Raspberry Pi 3 Model B bruger en Quad Core, som svarer til fire kerner. Derfor er TFLite også konfigureret med num_threads=4, så alle fire kerner udnyttes af modellen til at køre objekt detektion med maksimal ydeevne. Derudover har vi sat max_results=3, hvilket vil sige at den detekter højest 3 objekter hver eneste frame, og den ignorerer de øvrige. Score_threshold=0.3 gør at objekter med en konfidens under 30% sorteres fra, så Pi'en ikke bruger unødvendig processorkraft på at behandle objekt detektioner.

¹⁹ <https://www.kaggle.com/models/tensorflow/efficientdet/tensorFlow2/lite0-detection>

²⁰ <https://www.kaggle.com/datasets/awsaf49/coco-2017-dataset>

```

24 class ObjectDetector:
25
26     def __init__(self, model_path=MODEL_PATH, num_threads=4, score_threshold=0.3, max_results=3):
27         # Initialize the object detection model
28         base_options = core.BaseOptions(
29             file_name=model_path, use_coral=False, num_threads=num_threads)
30         detection_options = processor.DetectionOptions(
31             max_results=max_results, score_threshold=score_threshold)
32         options = vision.ObjectDetectorOptions(
33             base_options=base_options, detection_options=detection_options)
34         self.detector = vision.ObjectDetector.create_from_options(options)

```

4.5.3 Autonomous Mode (Nicklas)

En af udvidelserne i projektet var en autonomous mode, som gjorde det muligt for roveren at bevæge sig autonomt. Formålet var at lade roveren navigere selvstændigt og undgå forhindringer ved hjælp af ultralydssensoren og logikken

For at gøre roveren i stand til at undgå forhindringer brugte vi HC-SR04 ultralydssensoren som løbende målte afstanden til objekter foran. Målingerne blev brugt til at bestemme, hvordan roveren skulle reagere.

roveren kørte fremad og når den registrerede en genstand inden for sikkerhedsafstanden, stoppede den og drejede derefter til højre indtil der ikke var nogen forhindring, hvorefter den kørte fremad igen imens den så småt “auto correcter” til venstre og højre.

```

case AUTO_CORRECT_LEFT:
    // Turn slightly left to return closer to original di
    motors.turnLeft(TURN_SPEED);

    if (now - autoStateStartTime >= CORRECT_LEFT_TIME) {
        autoState = AUTO_FORWARD;
        autoStateStartTime = now;
    }

```

Under test observerede vi at roveren afveg fra den oprindelige rute ved at dreje til højre. For at

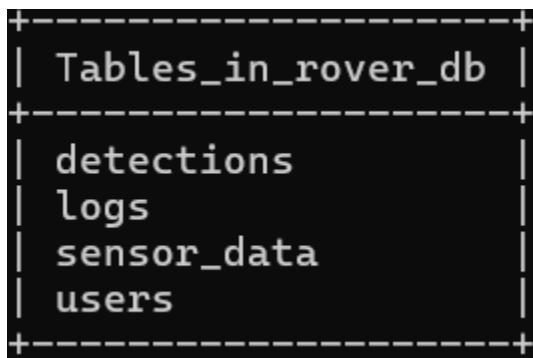
kompensere for det tilføjede vi “auto correct”. Efter den passere en forhindring laver den et drej til venstre, hvorefter den ender tilbage i normal automode. Dette hjælper den med at holde den originale kørselsretning.

4.6 Database (Nicklas)

4.6.1 MariaDB

Til projektet har vi anvendt MariaDB til lagring af roverens data. Databasen blev brugt til at gemme måle data fra sensorerne samt object identifikation fra kameraet. Formålet var at vise data i realtid på app'en men samtidig gemme data i databasen som kan anvendes til analyse.

Databasen blev opbygget med flere tabeller

A terminal window with a black background and white text. It shows the command 'Tables_in_rover_db' followed by a list of four tables: 'detections', 'logs', 'sensor_data', and 'users'. The text is enclosed in a dashed border with '+' characters at the corners.

Tables_in_rover_db
detections
logs
sensor_data
users

Tabellen sensordata blev brugt til at gemme temperatur, fugtighed og afstands målinger. Disse data blev sendt fra raspberryPi'en til backend gennem websocketten og derefter gemt i databasen ved hjælp af SQLAlchemy og SQL queries.

Tabellerne er opbygget af kolonner og unique id'er hvor hver række havde en primary key

sensordata tabellen indholdte fx. Felter som:

id	temperature	humidity	distance	created_at
1	NULL	NULL	NULL	2026-05-25 00:36:21
2	NULL	NULL	NULL	2026-05-25 00:36:23
4279	22.2	53.5	105.37	2026-05-25 19:50:04
4280	22.2	53.4	102.51	2026-05-25 19:50:06
4281	22.2	53.4	122.91	2026-05-25 19:50:08

Her fungerede id som primary key imens created_at blev brugt til at registrere tidspunkter

Tabellen detections blev brugt til at gemme objekt detections fra AI-modellen, her brugte vi objektets navn, farve og confidence score. Ved hjælp af sql queries kunne data sorteres og gemmes i databasen

id	object_name	color	confidence	image_path	created_at
441	color_detection	blue	NULL	NULL	2026-05-25 17:27:40
442	color_detection	blue	NULL	NULL	2026-05-25 17:28:00
672	color_detection	yellow	NULL	NULL	2026-05-25 19:16:47
673	color_detection	red	NULL	NULL	2026-05-25 19:16:54
674	color_detection	blue	NULL	NULL	2026-05-25 19:17:18

```
db.execute(  
    text("""  
    INSERT INTO sensor_data (  
        temperature,  
        humidity,  
        distance  
    )  
    VALUES (  
        :temperature,  
        :humidity,  
        :distance  
    )  
    """),  
    {  
        "temperature": temperature,  
        "humidity": humidity,  
        "distance": distance  
    }
```

Sql queries

For at reducere unødvendige database-skrivninger blev sensor-data ikke logget hvert sekund. I stedet blev live data sendt løbende til dashboardet, mens database-logging blev udført med længere intervaller. Dette reducerede belastningen på databasen og gjorde systemet mere effektivt.

For at undgå at databasen blev fyldt med gentagne object detections, blev der implementeret et cooldown-system, så samme objekt ikke blev registreret konstant flere gange i træk

Implementeringen af databasen gjorde det muligt at gemme og strukturere roverens data på en stabil og overskuelig måde. Under udviklingen opstod der problemer med null værdier, men de fleste af disse blev løst ved forkert end point, validering af data og cooldown logik. Vi nåede dog ikke at løse null problemet med object identifikation.

5. Dokumentation af udviklingsproces

5.1 Implementering og fejlretning (Nicklas)

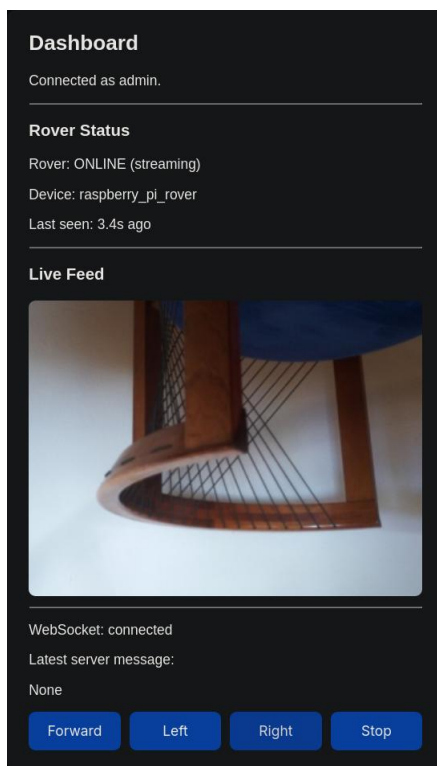
Manuel styring via server

Den manuelle styring af roveren blev håndteret via kontrolpanelet, hvor brugeren kan styre roveren via knapper. For at matche den oprindelige rovers styredata valgte vi at gøre brug af joystick værdier.

Under udviklingen opstod der problemer med håndteringen af joystickdata, da joystickdata fra rover og robotarmen ikke var adskilt ordentligt. Hvilket gjorde at kommandoerne kunne påvirke de forkerte funktioner. Hvilket betød at nogle gange styrede det roveren, imens robotarmen ikke virkede og omvendt.

Løsningen på problemet var at ændre datastrukturen så roveren og robotarmens data blev opdelt i separate felter. Sådan at frontend nu sender individuelle værdier som fx roverX, roverY, roverSW.

Efterfølgende har den nye datastruktur gjort så den manuelle styring er stabil og kan både styre robotarmen og roveren samtidig.



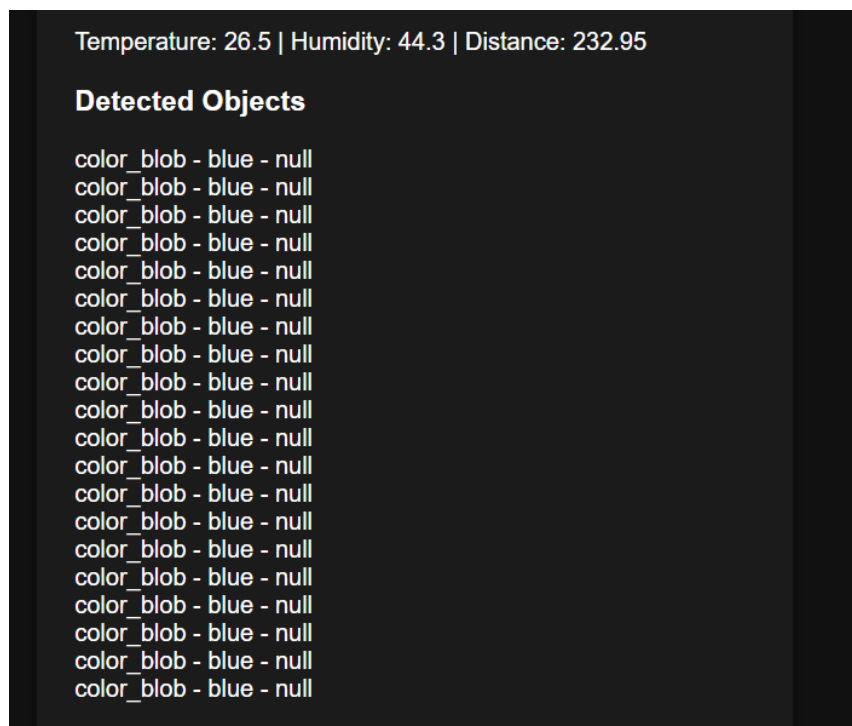
Farvegenkendelse og videostream

Under udviklingen af VideoStream observerede vi at billedet havde blå nuancer hvilket påvirkede farvegenkendelsen som kun kunne identificere blå. Efter fejlsøgning af videobehandlingen kunne vi konstatere at den brugte et forkert format. OpenCV arbejder med BGR formattet imens Videostreamen var sat til RGB format. Efter vi havde ændret formattet til

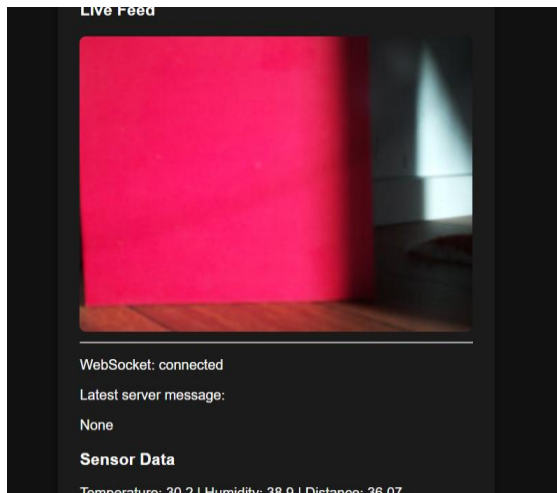
BGR i camera_manager.py, video_stream.py, color_detection.py og object_detection.py forsvandt de blå nuancer og farvegenkendelse kunne identificere flere farver.

Vi observerede under test at videostreamen var vendt på hovedet, dette blev rettet i frontend, hvor vi ændrede på måden videostreamen blev vist på hjemmesiden.

Før rettelse



Efter rettelse



DHT22 målinger

I det gamle projekt var DHT sensoren kun sat til at måle temperatur. Vi har efterfølgende udvidet funktionaliteten sådan, at den også kan måle luftfugtighed. Efterfølgende test har vist at temperatur og luftfugtigheds målinger er korrekte

underudviklingen var der et problem med, at sensordataen ikke blev vist i kontrolpanelet på webapplikationen og istedet viste null værdier. Efter fejlfinding fandt vi ud af at endpointet ikke var korrekt.

Efter rettelse af endpointet kunne sensordata ses i frontend og blev samtidig sendt til databasen.

Obstacle avoidance

Obstacle avoidance blev udført ved hjælp af HC-SR04 ultrasonic sensoren. Under test viste det sig at den kunne registrere forhindringer og stoppe foran dem samt undvige forhindringerne. I nogle tilfælde kunne roveren sætte sig fast i hjørner og have svært ved at finde en ny retning efter

undvigemanøvre eller autocorrect left.

Dette skyldes den simple navigations logik der bliver brugt i funktionen, som er baseret på undvig højre, autocorect venstre. Sensorens placering foran gør det også svært at registrere forhindringer når den drejer.

For at forbedre funktionens evne til at undgå forhindringer, satte vi obstacle distance op fra 25 til 35. Dette gav roveren mere tid til at reagere og mindskede risikoen for sammenstød. Ændringen forbedrede stabiliteten, men fjernede ikke problemet helt med at roveren satte sig fast.

AI objektgenkendelse

Farve genkendelse blev tidligere beskrevet færdigt udviklet og fungerede stabilt. Men objekt genkendelse blev kun delvist færdiggjort. Ved test observerede vi at objekter blev vist som null værdier i webapplikationens kontrolpanel samt databasen.

En mulig årsag til problemet kunne være håndteringen af data i main.py. Frameprocessor sendte værdierne "label" og "score" men main.py og resten af systemet forventede object og confidence. Der kan derfor have været fejl i oversættelsen fra frameprocessor til main.py, hvilket resulterede i ugyldige værdier.

På grund af projektets tidsramme prioriterede vi at starte på rapporten frem for at fortsætte yderligere fejlfinding.

Under udviklingen af backend & frontend, havde vi en del problemer med routing, portnumre og endpoints, specielt efter vi begyndte at lægge systemet bag Nginx reverse proxy. Backend kørte

på port 18765 men det skulle ikke være den port brugeren tilgik direkte. I stedet satte vi Nginx på port 50325 og så blev vores endpoint `/rover/` proxied videre til API'ens docker-container på `exam-backend:18765`.

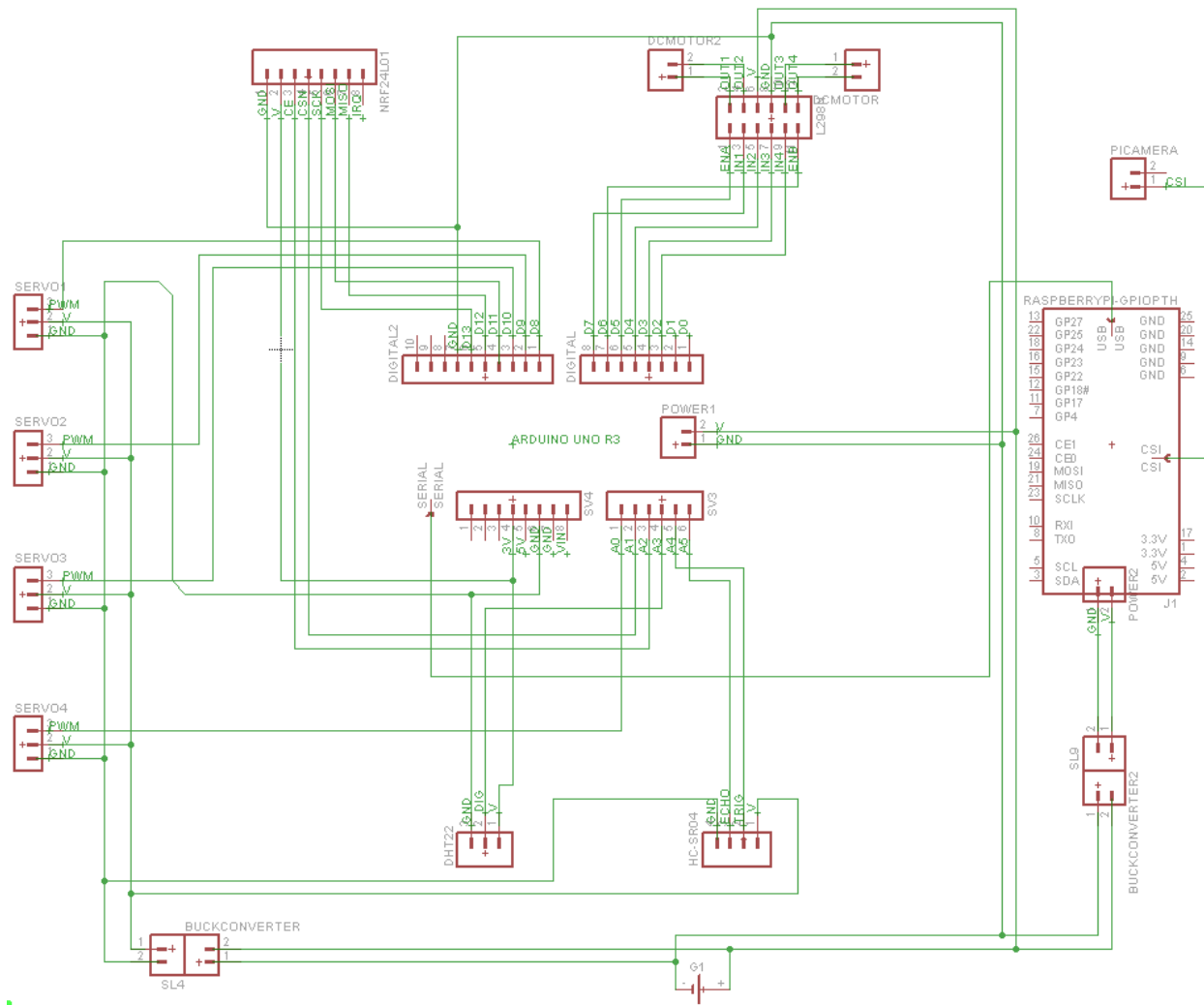
I starten gav det en masse fejl, fordi frontend og backend ikke kunne snakke sammen længere. Frontend kaldte stadig endpoints, som om der ikke var reverse proxy på, som fx `/api/sensor-data`, men når hele siden ligger under `/rover/`, skulle browseren ændres til at ramme `/rover/api-sensor-data`, ellers rammer requests forkert path og vil ikke fungere korrekt.

Der var også bøvl med trailing slash og redirects. Fx kunne `/rover` uden slash redirecte forkert og tabe porten 50325, og `/rover/api/rover/status` kunne redirecte til en path hvor `/rover` forsvandt. Sådan nogle småting kunne faktisk bryde hele flowet, fordi frontend spørger den forkerte URL.

Websocket-delen skulle også fixes mere specifikt fordi styringen ikke bare er almindelig http. Vi har to forskellige websockets, `/ws/control` og `/ws/rover`. Bag nginx skulle de derfor rammes som `/rover/ws/control` og `/rover/ws/rover`. Nginx skulle også sættes op med `proxy_http_version 1.1` og `"Connection upgrade"`, ellers blev forbindelsen ikke rigtigt opgraderet til websocket. Vi testede også at `/rover/ws/control` uden token blev afvist, mens den med login-token gav 101 Switching Protocols, og backend loggede at websocket-forbindelsen blev accepteret.

Der var også ændringer omkring login og database. Projektet startede meget simpelt, men blev senere flyttet over på MariaDB, hvor users, sensor-data, logs og detections blev gemt i tabellen. Login blev også gjort mere rigtigt med password hashing i stedet for bare at have et hardcoded login.

5.2 Hardwareopsætning (Nicklas)



5.3 Objektorienteret Programmering (Nicklas)

I roverprojektet har vi valgt at opbygge Rover softwaren objektorienteret. Hvilket vil sige, at systemet ikke er skrevet som en stor fil, hvor alt funktionalitet ligger samlet, men i stedet er opdelt i flere klasser. Hver klasse har ansvar for en bestemt del af roveren, fx. motorstyring, radiokommunikation, robotarm eller sensorer.

Formålet med denne opdeling har været at gøre koden mere overskuelig, lettere at fejlfinde i og nemmere at videreudvikle. Da roveren består af flere forskellige komponenter, ville en samlet kodefil hurtigt blive svær at læse og vedligeholde. Ved at bruge klasser kan hver komponent behandles som et selvstændigt objekt med egne data og funktioner.

Fx. håndteres alt, der har med motorerne at gøre, i klassen MotorController, mens robotarmen håndteres i RobotArm, og radiokommunikation håndteres i RadioReceiver. På den måde bliver hovedprogrammet mere simpelt, fordi det kun skal kalde funktioner som `motors.drive()`, `arm.control()` og `radio.readData()`, uden at kende alle de tekniske detaljer bag.

Klasser

Noget af det centrale i vores OOP-struktur, er at det er opbygget i klasser, hvor hver komponent har sin egen klasse. En klasse er en skabelon for et objekt, hvor variabler og funktioner samles.

I vores projekt bruges klasser bl.a. til:

<u>klasser</u>	<u>ansvar</u>
MotorController	styrer motorer, hastighed, retning, acceleration, stop
RobotArm	styrer servoer
RadioReceiver	modtager styrer data via nrf24L01 og sender temperatur tilbage.
TemperatureSensor	måler temperatur og luftfugtighed
UltrasonicSensor	måler afstand og registrere forhindringer foran

Det betyder, at hver klasse kun har ansvar for sin egen del af systemet. Det gør koden lettere at forstå, da fx. Alt motorlogik findes i MotorController, i stedet for en stor hovedfil, hvor man skal lede efter flere motor relaterede funktioner.

Encapsulation

En vigtig del af OOP i vores projekt er Encapsulation, hvor hver klasse skjuler sine interne funktioner og viser kun de ting frem resten af programmet skal bruge.

Et godt eksempel er MotorController. I header filen har klassen private variabler til motorpins, nuværende motorhastighed og indstillinger for maksimal hastighed og acceleration.

```
class MotorController : public Component {  
private:  
    int ENA, IN1, IN2;  
    int ENB, IN3, IN4;  
  
    int currentLeftMotor = 0;  
    int currentRightMotor = 0;  
  
    int maxSpeed;  
    int accelStep;  
  
    int rampMotor(int currentSpeed, int targetSpeed);  
    void setMotorA(int speedVal);  
    void setMotorB(int speedVal);  
};
```

Disse variabler og funktioner er private og kan derfor kun bruges i MotorController klassen. Hoved programmet kan ikke ændre på pins eller kalde setMotorA eller setMotorB. I stedet bruges det offentlige funktioner.

```

public:
    MotorController(
        int ena,
        int in1,
        int in2,
        int enb,
        int in3,
        int in4,
        int maxSpeed,
        int accelStep
    );

```

Det gør koden mere sikker og overskuelig. Hovedprogrammet skal kun fortælle MotorControlleren hvad roveren skal gøre fx. Motors.drive(...) eller motors.stop(). Detaljer som PWM, retningen på H-broen eller acceleration bliver håndteret internt af klassen.

Inheritance

I projektet gør vi også brug af enheritance det sker igennem header filen component.h hvor vi har defineret en baseklasse.

```

#ifndef COMPONENT_H
#define COMPONENT_H

class Component {
public:
    virtual void begin() = 0;
};

#endif

```

Det betyder at den ikke bruges som et objekt men en baseklasse de andre klasser kan arve fra. Funktionen begin() er en virtuel funktion hvilket betyder at alle klasser der arver fra den skal implementere deres egen begin(). På den måde sikre vi at alle komponenterne har deres egen initialiserings funktion, men selv bestemmer hvad der skal ske under opstart.²¹

²¹Geeksforgeeks. Virtual function in C++ (2026)
<https://www.geeksforgeeks.org/cpp/virtual-function-cpp/>

Klasserne har samme grund struktur og kan derfor initialiseres på en ensartet måde i hovedprogrammet og dette gør det mere læsbart

```
motors.begin();
arm.begin();
radio.begin();
temperatureSensor.begin();
ultrasonic.begin();
```

Abstraction

MotorController indeholder også abstraction. Abstraction betyder at tekniske detaljer samles under en struktur.

Brugeren styrer roveren via et joystick som sender værdierne 0-1023. I MotorController bliver disse værdier omsat til hastighed.

```
int x = map(xRaw, 0, 1023, -maxSpeed, maxSpeed);
int y = map(yRaw, 0, 1023, -maxSpeed, maxSpeed);
```

Derefter håndteres deadzone så små udsvinger omkring joystickets midter position ikke får roveren til at bevæge sig.

```
if (abs(x) < deadzone) x = 0;
if (abs(y) < deadzone) y = 0;
```

Til sidst beregnes venstre og højre motorværdi

```
int targetLeft = constrain(y + x, -maxSpeed, maxSpeed);
int targetRight = constrain(y - x, -maxSpeed, maxSpeed);
```

Hovedprogrammet ser kun motors.drive(...) men i klassen bliver joystick input beregnet og begrænset og sendt videre til motorerne. Det gør teksten mere læsbar.

RaspberryPi

Vores raspberryPi er også opdelt i klasser:

<u>Klasse</u>	<u>Ansvar</u>
ArduinoSerial	Sender kontrolkommandoer til Arduino og læser sensordata

ServerClient	Håndterer WebSocket-forbindelse til serveren
CommandHandler	Oversætter serverkommandoer til Arduino-kommandoer
CameraManager	Starter kameraet og henter frames
VideoStream	Sender MJPEG-stream over HTTP
FrameProcessor	Behandler billeder og samler resultater
ObjectDetector	Kører objektgenkendelse med TensorFlow Lite
ManualControl	Opretter joystick-lignende kommandoer

Dette gør det muligt at holde billedbehandling, kommunikation og kamera adskilt fra hinanden. Hvis der fx. skal ændres i websocket kommunikation sker det i ClientServer. Eller hvis der skal ændres i billedbehandlingen, sker det i FrameProcessor

Ved at bruge objektorientet programmering er vores kode blevet mere struktureret og lettere at vedligeholde. Hver komponent i roveren har sin egen klasse, hvilket gør det lettere at forstå og videreudvikle.

Inheritance bruges igennem component baseklassen, hvor flere klasser arver begin() funktionen.

Encapsulation bruges til at skjule hardware detaljer som pins, PWM, servopositioner m.m.

Abstraction gør at hovedprogrammet kan kalde funktioner som fx. motor.drive(...) istedet for at skulle styre hver pin.

Samlet set har OOP gjort det muligt at opbygge roveren modulært, hvor arduino og raspberry arbejder sammen gennem adskilte software komponenter.²²

²² Microsoft Learn. Object oriented programming (C#)
<https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/tutorials/oop>

5.5 Brug af AI i projektet (Alle)

I projektet har vi brugt AI som et hjælpeværktøj undervejs. Det har primært fungeret som en form for sparring, når vi sad fast med fejl som var svære at finde ift. Arduino, Raspberry Pi, frontend og backend, Docker, Nginx.

AI blev anvendt som en sparringspartner under udviklingen af flere af funktionerne. Vi stod selv for de overordnede design valg bl.a. websocket kommunikation og serial kommunikationen mellem RaspberryPi og Arduino, mens AI har bidraget med forslag til implementering og struktur.

AI blev desuden brugt som et læringsredskab til at forstå nogle tekniske koncepter samt kode. Dette gjorde det lettere at sætte sig ind i dele af systemet vi ikke havde udviklet helt fra bunden.

Derudover har vi brugt Codex som en del af debugging-processen. Codex blev sat op med sin egen bruger, som kun havde read-only adgang til en bestemt projektmappe, hvilket betød at Codex ikke kunne ændre vores filer, men kunne analysere og finde fejl.

6. Konklusion (Alle)

6.1 Erfaringer

Projektet har samlet set været et ret stort projekt fordi det ikke kun handlede om en ting. Vi skulle arbejde med hardware, programmering, netværk, server, database, frontend, backend, kamera-stream, Ai. Det har gjort projektet spændende at arbejde med, men også ret komplekst.

Noget af det der fungerede bedst i projektet, var opdelingen af vores forskellige komponenter, altså Arduino Uno, Raspberry Pi 3 og serveren. Arduinoen stod for de mere direkte hardwareopgaver som motorstyring, robotarm, sensorer og autonomous mode. Pi'en stod for at være mellemlid mellem selve roveren og serveren, håndtering af kamera, AI og websocket. Serveren stod for backend, API, database og frontend. Hver del med sit eget ansvar.

6.2 Diskussion

- Hvad fungerede godt

Selve opbygningen af vores rover er gået meget godt. Ved at kombinere RaspberryPi og arduino blev opgaverne fordelt imellem de 2 enheder arduino styrede sensorer, motorer og servoer imens RaspberryPi tog sig af videostream, AI, og kommunikation med serveren. Dette gjorde det mere overskueligt og hjalp med at få systemet til at køre bedre.

Efter problemerne var løst med fjernstyringen fungerede det rent faktisk meget godt. Brugeren kunne styre både robotarmen og roverens bevægelser via kontrolpanelet i webapplikationen og kommunikationen mellem frontend, backend, raspberrypi og arduino stabilt

live videofeedet fungerede godt og gav brugeren mulighed for at se ud fra roverens perspektiv i realtid. Ved at videofeedet blev håndteret separat fra styringsdataen, hvilket resulterede i en mere stabil løsning.

- Hvad kunne forbedres

Objektgenkendelsen nåede ikke at blive udviklet lige så langt som farvegenkendelsen. Da der var udfordringer imellem frameprocessor og resten af systemet. Pga. Tidsrammen for projektet blev det ikke færdigudviklet.

Den Autonome kørsel virkede egentlig generelt tilfredsstillende, men roveren kunne nogle gange sætte sig fast når den drejede og der var en forhindring i vejen, typisk under “autocorrect left” funktionen da ultrasonic sensoren er placeret foran.

Farvegenkendelsen krævede flere justeringer rundt om i systemet. Bl.a. skulle RGB formatet ændres til BGR i billedbehandlingen, hvilket der blev brugt længere tid på at fejlfinde.

Cloudflare kunne også være en forbedring. Vi testede det, men nåede ikke at gøre det til en fast del af projektet. Cloudflare kan bruges til at give mere sikker adgang via HTTPS.

- Begrænsninger i projektet

Flere funktioner kunne have været videre udviklet og optimeret igennem test og fejlfinding. Projektets tidsramme betød, at nogle funktioner blev prioriteret højere end andre.

Da ultrasonicsensoren er forbundet til samme strøm, som servoerne, kan brug af armen sammen med autonomous mode ødelægge sensormålingerne pga. Strømsving og fx. Derfor få roveren til at registrere en forhindring der ikke er der, eller ikke registrere en forhindring der er der.

RaspberryPi 3 havde nogle begrænsninger for vores projekt ift brug af AI, hvilket gjorde at vi valgte OpenCV som er en library der dækker vores behov, og TFLite der er optimeret til at have

6.3 Fremtidige forbedringer

Vores nuværende roverne har kun en ultrasonic sensor monteret, hvis vi havde tilføjet nogen på siderne af roveren, ville den kunne registrere forhindringer i flere retninger. Dermed kunne bedre navigationsvalg træffes.

Den nuværende obstacle avoidance reagere kun på forhindringer i realtid. I en forbedret version kunne vi anvende nogle mere avanceret navigations algoritmer, som planlægger ruter og husker tidligere registrerede forhindringer, fx SLAM.

ved at gøre brug af objektgenkendelse og ved hjælp af robotarmen kunne roveren samle identificerede objekter op uden bruger styring.

6.4. Konklusion

Vi kan konkludere at projektet samlet set har vist, at det var muligt at videreudvikle den gamle rover til et system, hvor den kan styres via en webapplikation, sende sensordata som vises på applikationen og kommunikere med en backend og database.

Vi har lavet en løsning hvor Arduinoen håndterer hardware, Raspberry Pi fungerer som mellemlid og hvor serveren håndterer frontend, backend og database. Integrationen af

RaspberryPi, ultrasonic sensor og kamera har samtidig gjort det muligt at udvide roveren med autonome funktioner, videostreaming og AI.

Alt blev ikke perfekt, objektgenkendelsen havde problemer, autonomousmode kunne sætte sig fast i nogle situationer, hjemmesiden kunne sikres bedre og præsenteres mere brugervenligt. Men de vigtigste dele af systemet fungerer og vi fik en god forståelse for hvor svært det kan være at få så mange forskellige komponenter/teknologier til at snakke sammen. Projektet er derfor ikke kun resulteret i en funktionel rover, men også givet os erfaring med integration af hardware, software, netværks kommunikation og AI.

7. Kildeliste:

Websider

Docker Documentation. (n.d.). “*Docker Compose.*”

<https://docs.docker.com/compose/>

Docker Documentation. (n.d.). “*Docker overview.*”

<https://docs.docker.com/get-started/docker-overview/>

Docker Documentation. (n.d.). “*Networking in Compose.*”

<https://docs.docker.com/compose/how-tos/networking/>

FastAPI Documentation. (n.d.). “*WebSockets.*”

<https://fastapi.tiangolo.com/advanced/websockets/>

GeeksforGeeks. (2026) “*Virtual function in C++.*”

<https://www.geeksforgeeks.org/cpp/virtual-function-cpp/>

Microsoft Learn. (2025) “*Object-oriented programming (C#)*”

<https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/tutorials/oop>

Mannaz. (2026) *Hvad er scrum? En introduktion til Scrum metoden.*

<https://www.mannaz.com/da/artikler/projektledelse-agile-principper/hvad-er-scrum-en-introduktion-til-scrum-metoden/>

MDN Web Docs. (2025) “*The WebSocket API.*”

https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API

NGINX Documentation. (n.d.). “*NGINX Reverse Proxy.*”

<https://docs.nginx.com/nginx/admin-guide/web-server/reverse-proxy/>

NGINX Documentation. (n.d.). “*WebSocket proxying.*”

<https://nginx.org/en/docs/http/websocket.html>

Kaggle. (2020). *Tensorflow/efficientdet model.*

<https://www.kaggle.com/models/tensorflow/efficientdet/tensorFlow2/lite0-detection>

Uvicorn Documentation. (n.d.). “*Uvicorn.*”

<https://uvicorn.dev/>

OpenCV. (n.d.). *Introduction.*

<https://docs.opencv.org/4.x/d1/dfb/intro.html>

OpenCV. (n.d.). *Contours : Getting Started.*

https://docs.opencv.org/4.x/d4/d73/tutorial_py_contours_begin.html

Kaggle. (2020). *COCO 2017 DATASET*.

<https://www.kaggle.com/datasets/awsaf49/coco-2017-dataset>

pySerial API. (n.d.). *pySerial 3.5 documentation*.

https://pyserial.readthedocs.io/en/latest/pyserial_api.html

Developer Mozilla Network (MDN), WebSocket API(WebSockets)

https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API

(Accessed: Jun 1. 2026)

8. Bilag:

Bilag A – Scrum-sprints dokumentation (vedhæftet i separat PDF)

Bilag B - Backlog (vedhæftet i separat PDF)

Bilag C - Github - <https://github.com/Klaes25/examproject>